

ЧЕРНІВЕЦЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ
ІМЕНІ ЮРІЯ ФЕДЬКОВИЧА

КАФЕДРА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ
КОМП'ЮТЕРНИХ СИСТЕМ

**КУРСОВИЙ ПРОЄКТ:
СТРУКТУРИ ДАНИХ, АЛГОРИТМИ
ТА ОБ'ЄКТНО-ОРІЄНТОВАНА ПАРАДИГМА**

на тему: Система управління базою
замовлення страв в ресторані

здобувач вищої освіти II курсу 243-Б групи
спеціальності 121 «Інженерія програмного
забезпечення»
першого (бакалаврського) рівня вищої освіти
Малофій А.Е.

Оцінка:

за національною шкалою

(словами)

кількість балів

(цифра)

за шкалою ECTS

(літера)

Чернівці, 2025

ЧЕРНІВЕЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ЮРІЯ ФЕДЬКОВИЧА

Навчально-науковий інститут фізико-технічних та комп'ютерних наук
Кафедра програмного забезпечення комп'ютерних систем

ЗАВДАННЯ
на курсовий проєкт
здобувачу вищої освіти
першого (бакалаврського) рівня вищої освіти
Малофія Артура Едуардовича

- 1) Тема проєкту: Розробка програмної системи управління базою замовлення страв у ресторані
- 2) Вхідні дані до проєкту: Проєктована система повинна забезпечувати автоматизацію процесів управління меню (страви та напої) та формування замовлень клієнтів. Об'єкти меню характеризуються категорією, назвою, ціною, вагою, наявністю та часом приготування (для страв) або об'ємом та вмістом алкоголю (для напоїв).
- 3) Вихідні дані до проєкту:
 - визначені сценарії використання програмної системи (авторизація, маніпуляція даними меню, формування замовлення);
 - розроблені блок-схеми роботи програми: узагальнений алгоритм роботи системи, алгоритми додавання, редагування, видалення та пошуку об'єктів;
 - задокументовано перелік класів, які реалізують основну бізнес-логіку: IEntity (інтерфейс), MenuItem (абстрактний клас), Dish, Drink, UserManager, Order, User, RestaurantManager;
 - описано функції програми на основі сценаріїв використання (пошук за категорією, розрахунок чека, фільтрація за ціною);
 - проведено функціональне тестування системи в умовах коректних та некоректних дій користувача (обробка виключень std::exception);
 - розроблена програмна документація з описом ієрархії класів та взаємодії модулів;
 - підготовлена презентація доповіді в електронному;

Керівники проєкту

_____ Валь О.О.
(підпис керівника)

(підпис керівника) Дяконенко Б.В.

(підпис керівника) Комісарчук В.В.

Завдання взяла до
виконання

(підпис студентки) Малофій А. Е.

РЕФЕРАТ

Курсовий проєкт налічує 50 сторінок, 43 рисунків, 7 таблиць та 6 джерел

Об'єктом проєктування є програмні сутності: базовий інтерфейс (IEntity), абстрактний клас (MenuItem), класи-моделі (Dish, Drink, User), класи-контролери (RestaurantManager, UserManager) та об'єкти замовлень (Order) .

Метою роботи є розробка програмної системи «Управління замовленнями ресторану» призначеної для автоматизації процесів управління меню (страви та напоїв), ведення обліку користувачів та формування клієнтських замовлень.

Для забезпечення ефективного зберігання даних обрано формат файлів CSV та TXT. Програма реалізована у вигляді консольного застосунку мовою C++ із застосуванням об'єктно-орієнтованого підходу (абстракція, поліморфізм, інкапсуляція).

Розроблений функціонал включає:

- систему авторизації з розмежуванням прав доступу (Адміністратор/Клієнт) ;
- повний цикл маніпуляції даними меню: додавання, редагування специфічних атрибутів (час приготування, об'єм, вміст алкоголю) та видалення об'єктів ;
- алгоритми пошуку за назвою / категорією та сортування за ціною ;
- механізм формування замовлення: додавання позицій до кошика, перегляд деталізованого чека та автоматичний розрахунок загальної вартості.

Система дозволяє централізувати дані закладу, мінімізувати помилки при розрахунках та підвищити оперативність обслуговування відвідувачів.

АВТОМАТИЗАЦІЯ РЕСТОРАНУ, ОБ'ЄКТНО-ОРІЄНТОВАНЕ ПРОГРАМУВАННЯ, ІЄРАРХІЯ КЛАСІВ, CSV-СХОВИЩЕ, ПОЛІМОРФІЗМ, C++, КОНСОЛЬНИЙ ДОДАТОК

SUMMARY

The coursework project consists of 50 pages, 43 figures, 7 tables and 6 sources.

The object of the design encompasses the software entities: the base interface (IEntity), the abstract class (MenuItem), model classes (Dish, Drink, User), controller classes (RestaurantManager, UserManager), and order objects (Order).

The goal of the work is to develop the «Restaurant order management» software, designed to automate menu management processes (dishes and drinks), user accounting, and customer order formation.

To ensure efficient data storage, CSV and TXT file formats were chosen. The program is implemented as a console application in C++ using an object-oriented approach (abstraction, polymorphism, encapsulation).

The developed functionality includes:

- an authorization system with access rights separation (Admin/Client);
- a full cycle of menu data manipulation: adding, editing specific attributes (cooking time, volume, alcohol content), and deleting objects;
- search algorithms by name/category and sorting by price;
- an order formation mechanism: adding items to the cart, viewing a detailed, and automatic calculation of the total cost.

The system allows centralizing restaurant data, minimizing calculation errors, and increasing the speed of customer service.

*RESTAURANT AUTOMATION, OBJECT-ORIENTED PROGRAMMING,
CLASS HIERARCHY, CSV STORAGE, POLYMORGHISM, C++, CONSOLE
APPLICATION*

ЗМІСТ

<u>ПЕРЕЛІК СКОРОЧЕНЬ ТА УМОВНИХ ПОЗНАЧЕНЬ</u>	<u>7</u>
<u>РОЗДІЛ 1. СПЕЦИФІКАЦІЯ ПРОЄКТУ</u>	<u>8</u>
<u>1.1. Постановка завдання</u>	<u>8</u>
<u>1.2. Опис вимог до програмної системи</u>	<u>8</u>
<u>1.3. Опис сценарії використання програми</u>	<u>9</u>
<u>РОЗДІЛ 2. ПРОГРАМНА ДОКУМЕНТАЦІЯ</u>	<u>15</u>
<u>2.1. Алгоритм розв'язання задачі</u>	<u>15</u>
<u>2.2. Проєктування програмної системи</u>	<u>26</u>
<u>2.3. Опис програмної системи</u>	<u>30</u>
<u>2.4. Тестування програмної системи</u>	<u>38</u>
<u>2.5. Програмні засоби розробки</u>	<u>48</u>
<u>ВИСНОВКИ</u>	<u>50</u>
<u>СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ</u>	<u>52</u>
<u>ДОДАТКИ</u>	<u>53</u>
<u>Додаток А. Скролінг програми</u>	<u>53</u>

ПЕРЕЛІК СКОРОЧЕНЬ ТА УМОВНИХ ПОЗНАЧЕНЬ

№	Скорочення	Пояснення
1	ІС	Information System - інформаційна система
2	ООП	Об'єктно-орієнтоване програмування
3	CSV	Comma-Separated Values - текстовий формат файлу для зберігання табличних даних
4	СД	Сховище даних (файли menu.csv та users.txt)
5	RTTI	Run-Time Type Information (механізм dynamic_cast)
6	CRUD	Create, Read, Update, Delete (основні операції з даними)

РОЗДІЛ 1. СПЕЦИФІКАЦІЯ ПРОЄКТУ

1.1. Постановка завдання

Об'єктом проєктування є програмна система «Restaurant Management System». Предметна область охоплює процеси автоматизації роботи закладу громадського харчування, що включають управління електронним меню (страви та напої), ведення обліку користувачів та оперативне формування клієнтських замовлень.

Основними учасниками системи є неавторизовані та авторизовані користувачі. Доступ до основних функцій програми надається лише після успішної авторизації. Система передбачає дві основні ролі:

- **Адміністратор:** має повні права на управління базою даних меню (додавання, редагування, видалення позицій) та управління акаунтами інших користувачів.
- **Клієнт (Користувач):** має доступ до перегляду меню, пошуку страв, сортування за ціною та формування власного замовлення з автоматичним розрахунком вартості.

1.2. Опис вимог до програмної системи

Вимоги до функціональності:

- **Управління меню:** збереження та обробка даних про страви (категорія, назва, ціна, вага, час приготування) та напоїв (об'єм, вміст алкоголю).
- **Робота із замовленнями:** створення замовлення шляхом додавання позицій до кошика, перегляд списку обраних страв, видалення позицій та повне очищення кошика.
- **Обробка даних:** реалізація механізмів пошуку за ключовим словом (назва або категорія) та сортування всього меню за зростанням ціни.
- **Фінансовий розрахунок:** автоматичне формування чека та обчислення загальної суми до оплати.

Вимоги до архітектури та інтерфейсу:

- **Мова та середовище:** програма реалізована мовою C++ у вигляді консольного додатка.
- **Збереження даних:** Сховище даних (СД) організоване у вигляді текстових файлів: menu.csv (база страв та напоїв) та users.txt (облікові записи користувачів).
- **Надійність:** обов'язкова перевірка коректності введення даних (валідація) та централізована обробка виключних ситуацій (exception handling) для запобігання критичним збоям програми.
- **Інтерфейс:** робоча мова - українська; навігація здійснюється за допомогою текстового меню.

1.3. Опис сценаріїв використання програми

Таблиця 1 - Сценарій 1

Сценарій 1: Формування замовлення клієнтом	
Передумова: Користувач авторизований з роллю «Клієнт» і відкрив головне меню.	
Основний сценарій	
Крок 1	Користувач обирає пункт «Додати страву до замовлення» та вводить ID обраної позиції.
Крок 2	Програма перевіряє наявність ID у базі menu.csv. Якщо об'єкт знайдено, він додається до списку Order.
Крок 3	Програма виводить повідомлення про успішне додавання та поточну кількість товарів у кошику.
Альтернативний сценарій	
Крок 1	Користувач вводить ID, якого не існує в системі.
Крок 2	Програма не знаходить відповідності у базі даних.
Крок 3	Користувач бачить «Помилка: ID не знайдено»

Таблиця 2 - Сценарій 2

Сценарій 2: Редагування даних адміністратором	
Передумова: Користувач авторизований з роллю «Адміністратор» та обирає пункт «Управління меню» .	
Основний сценарій	
Крок 1	Адміністратор обирає пункт «Редагування існуючої позиції» та вводить ID обраної позиції.
Крок 2	Програма зчитує та виводить поточні дані: назву, ціну, вагу, а також специфічні поля (категорія і час для приготування страв або об'єм та алкогольність для напоїв).
Крок 3	Програма пропонує ввести нові значення для кожного поля. Користувач вводить нові дані або натискає Enter, щоб залишити поточне значення без змін.
Крок 4	Система оновлює стан об'єкта в пам'яті та автоматично синхронізує зміни з файлом menu.csv, після виходу з програми.
Альтернативний сценарій	
Крок 1	Адміністратор обирає пункт «Редагування існуючої позиції» та вводить ID обраної позиції.
Крок 2	Адміністратор вводить ID, якого немає в базі даних.
Крок 3	Програма виводить повідомлення «Об'єкт з ID не знайдено» та повертає користувача до меню управління.
Крок 4	У разі введення некоректного типу даних (текст замість ціни), виводить помилку «Увага: некоректно введено ID!» та повертає у головне меню

Таблиця 3 - Сценарій 3

Сценарій 3: Універсальний пошук меню за категорією
Передумова: Користувач авторизований з роллю «Клієнт»

Основний сценарій	
Крок 1	Користувач обирає функцію «Пошук страв та напоїв» та вводить назву категорії.
Крок 2	Програма фільтрує список об'єктів за вказаним ключем без урахування регістру.
Крок 3	Користувач бачить перелік усіх страв, що належить обраної категорії.
Альтернативний сценарій	
Крок 1	Користувач обирає функцію «Пошук страв та напоїв» та вводить назву категорії.
Крок 2	Жодна страва не відповідає введеному запиту
Крок 3	Користувач бачить повідомлення: «За вашим запитом нічого не знайдено»

Таблиця 4 - Сценарій 4

Сценарій 4: Авторизація та розмежування прав доступу	
Передумова: Програма запущена, завантажено базу користувачів базу користувачів з users.txt, а також запитує логін та пароль.	
Основний сценарій	
Крок 1	Користувач вводить облікові дані.
Крок 2	Система перевіряє наявність пари логін/пароль у UserManager.
Крок 3	При успішному збігу програма перевіряє прапорець isAdmin.
Крок 4	Відкривається відповідне головне меню (Адмін-панель або Меню клієнта).
Альтернативний сценарій	
Крок 1	Користувач вводить облікові дані.
Крок 2	Система перевіряє наявність пари логін/пароль у UserManager.

Крок 3	Введено неіснуючий логін або неправильний пароль
Крок 4	Система виводить повідомлення: «Помилка! Неправильний логін та пароль.» та повертається на крок 1 (повторна спроба)

Таблиця 5 - Сценарій 5

Сценарій 5: Додавання нової позиції до бази даних (GRUD)	
Передумова: Користувач авторизований як адмін, вибрано обирає пункт «Управління меню»	
Основний сценарій	
Крок 1	Адміністратор обирає «Додати нову страву» або «Додати новий напій.»
Крок 2	Програма автоматично генерує унікальний ID.
Крок 3	Адміністратор вводить параметри (назва, ціна, вага категорія тощо)
Крок 4	Програма створює новий об'єкт у динамічній пам'яті та додає його до вектора menu.
Крок 5	Система виводить підтвердження про успішне додавання
Альтернативний сценарій	
Крок 1	Під час числових даних (ціна/вага/час/об'єм/місткість алкоголю)
Крок 2	Спрацьовує механізм try-catch, викликається функція ClearInput()
Крок 3	Виводиться попередження, операція переривається без пошкодження бази даних

Таблиця 6 - Сценарій 6

Сценарій 6: Розрахунок вартості та закриття замовлення (Чек	
Передумова: Користувач авторизований з роллю «Клієнт». У кошику наявна хоча б одна позиція.	

Основний сценарій	
Крок 1	Клієнт обирає пункт «Розрахувати вартість (Чек)».
Крок 2	Програма ітерує по списку замовлення, викликаючи CalculateTotal().
Крок 3	На екран виводиться деталізований список страв та напоїв, їх ціни та підсумкова сума
Крок 4	Клієнт підтверджує оплату.
Крок 5	Об'єкт Order очищує, виводить повідомлення «Чек оплачено» та повідомлення про знищення об'єкта.
Альтернативний сценарій	
Крок 1	Кошик порожній
Крок 2	При спробі перегляду чека програма виводить повідомлення «Кошик порожній!».
Крок 3	Повернення до меню клієнта

Таблиця 7 - Сценарій 7

Сценарій 7: Виключення страви або напою	
Передумова: Адміністратор авторизований, обрано пункт «Видаляти позицію за ID» в меню «Управління меню»	
Основний сценарій	
Крок 1	Програма запиту ID об'єкта для видалення.
Крок 2	Адміністратор вводить числовий ідентифікатор.
Крок 3	Система виконує пошук об'єкта у векторі menu за допомогою методу FindById.
Крок 4	Якщо об'єкт знайдено, програма видаляє його з динамічної пам'яті (delete) та вилучає вказівник з контейнера.
Крок 5	Виводить повідомлення про успішне видалення.
Альтернативний сценарій	
Крок 1	Адміністратор вводить ID, якого не існує.

Крок 2	Програма ідентифікує відсутність об'єкта (повертає nullptr).
Крок 3	Виводиться повідомлення: «Помилка: об'єкт з таким ID не знайдено».

РОЗДІЛ 2. ПРОГРАМНА ДОКУМЕНТАЦІЯ

2.1. Алгоритм розв'язання задачі

Узагальнена блок-схема алгоритму автентифікації та розмежування прав доступу користувачів наведено на рис. 1.

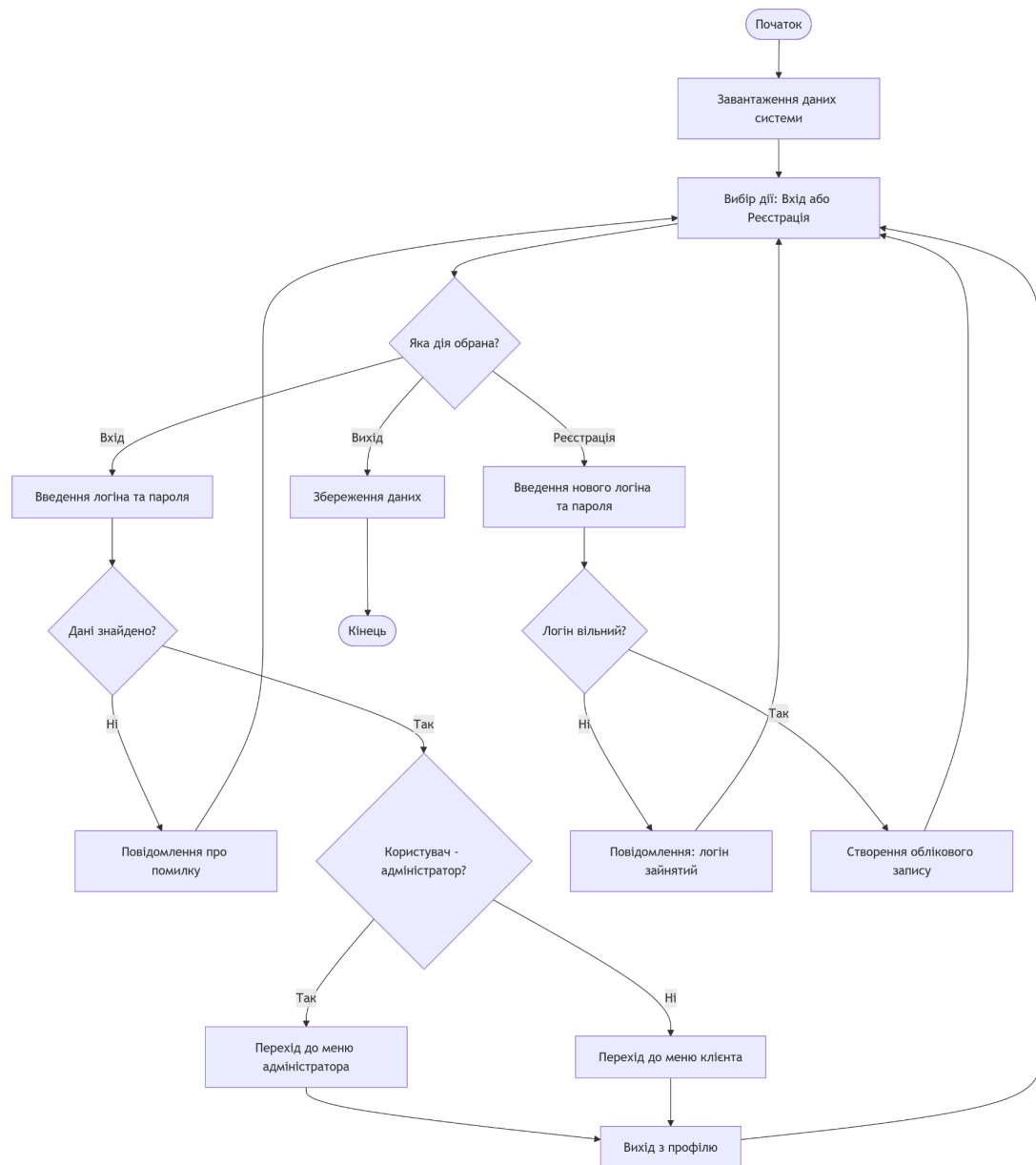


Рисунок 1 – Блок-схема алгоритму автентифікації та розмежування прав доступу .

Узагальнені блок-схеми алгоритмів функціонування головного циклу меню клієнта, формування замовлення з перевіркою типів позицій, а також універсального пошуку об'єктів у базі даних наведено на рис. 2-4

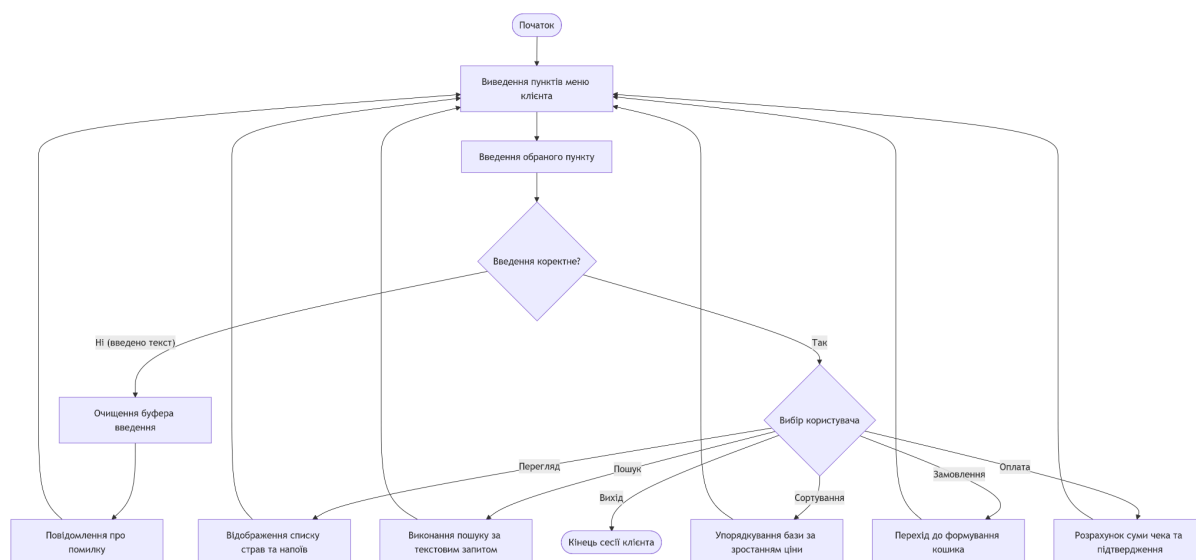


Рисунок 2 – Блок-схема алгоритму головного циклу меню клієнта .

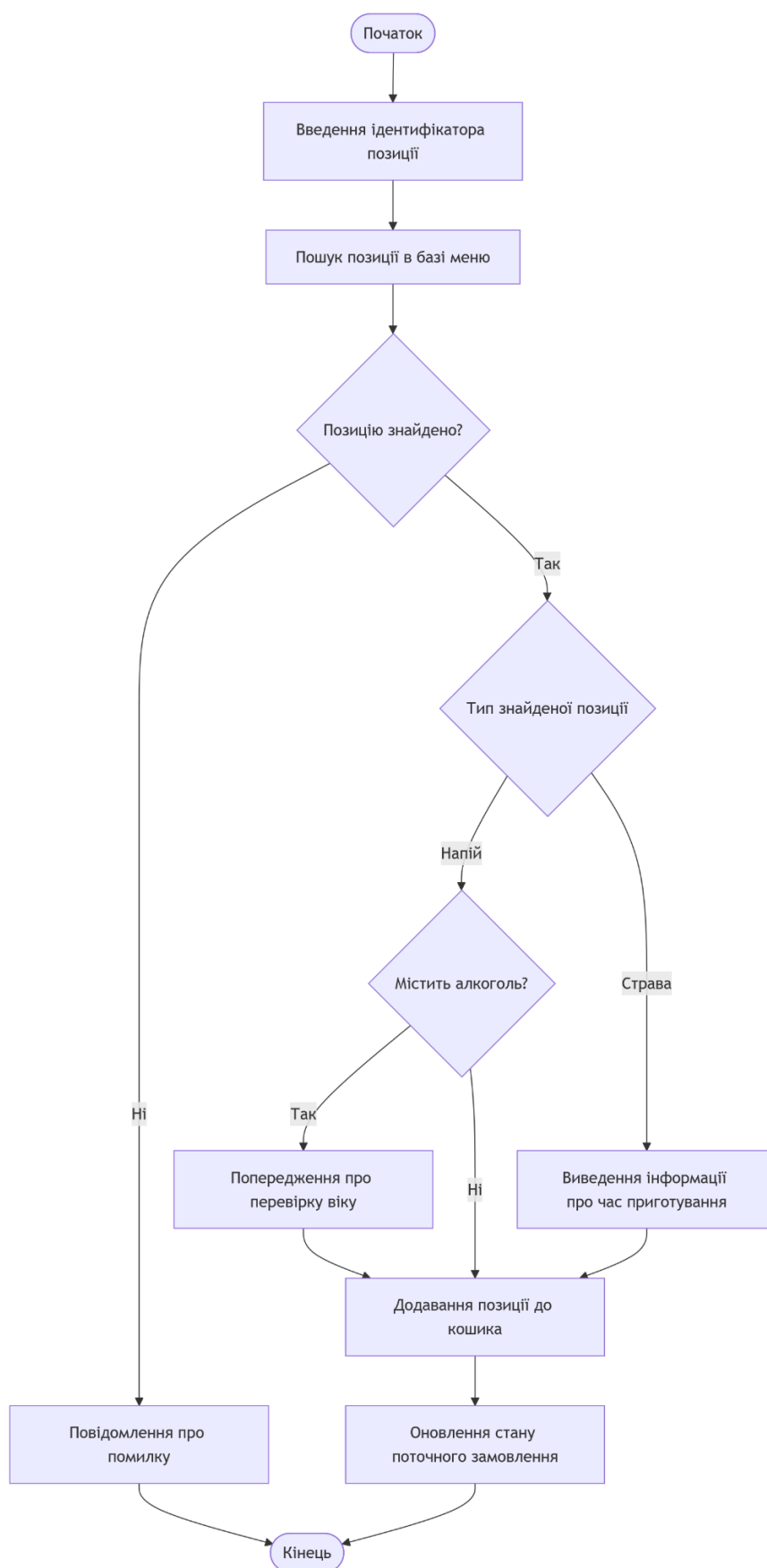


Рисунок 3 – Блок-схема алгоритму формування замовлення клієнтом

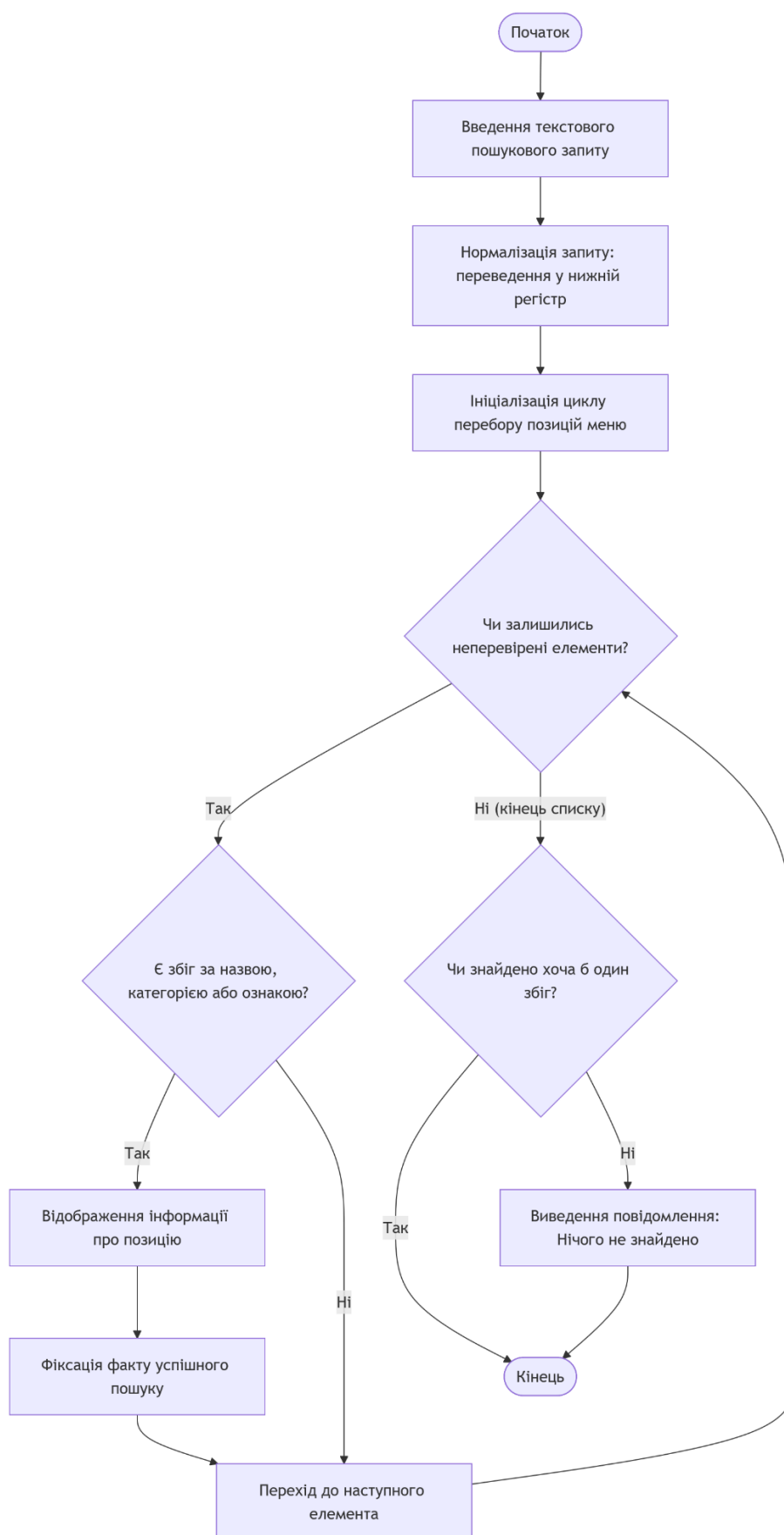


Рисунок 4 – Блок-схема алгоритму універсального пошуку в базі даних

Узагальнені блок-схеми алгоритмів функціонування підсистеми управління базою даних (меню адміністратора), зокрема процесів додавання нових об'єктів, редагування існуючих позицій та їх безпечного видалення з пам'яті, наведено на рис. 5 – 7.

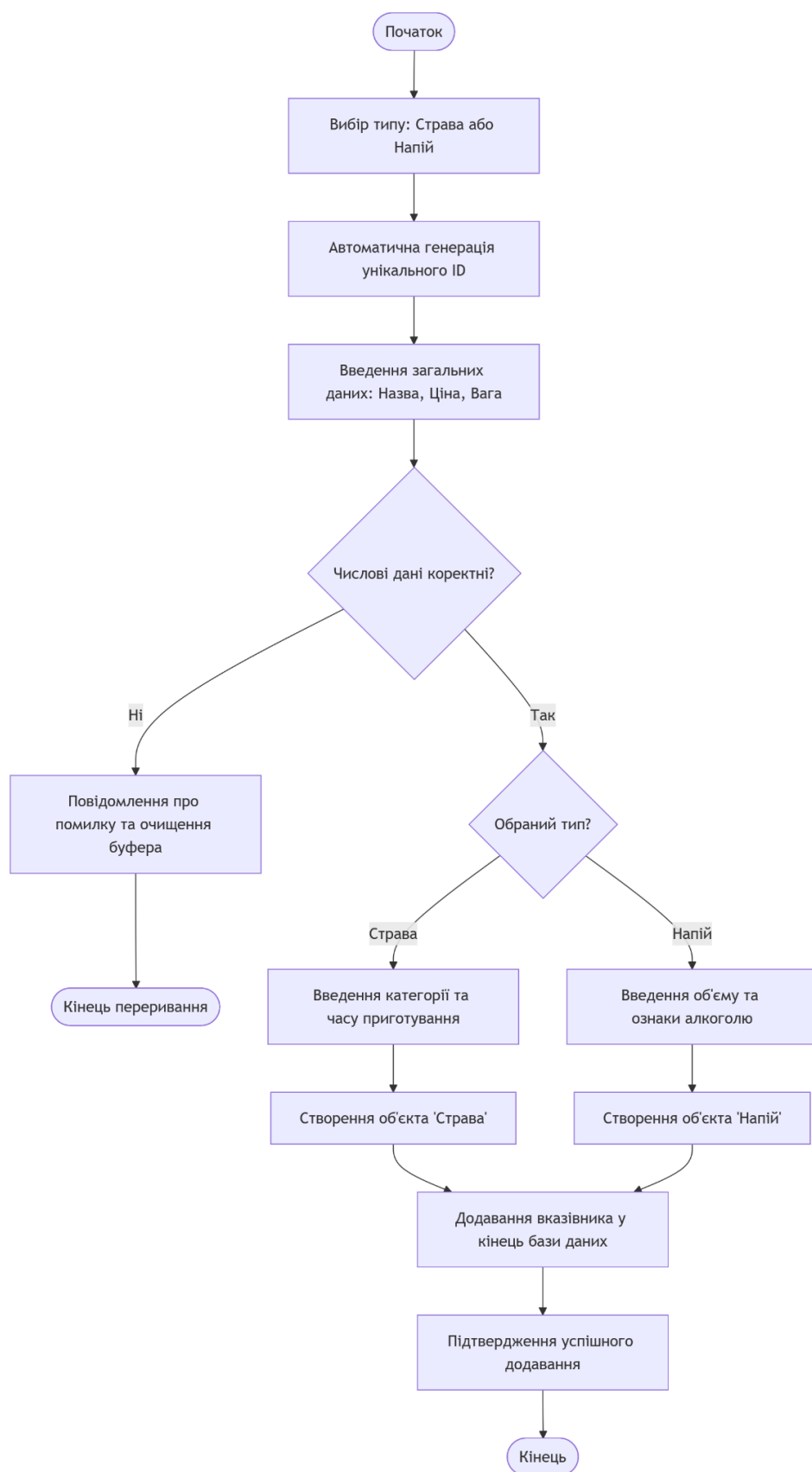


Рисунок 5 – Блок-схема алгоритму додавання нової позиції до меню

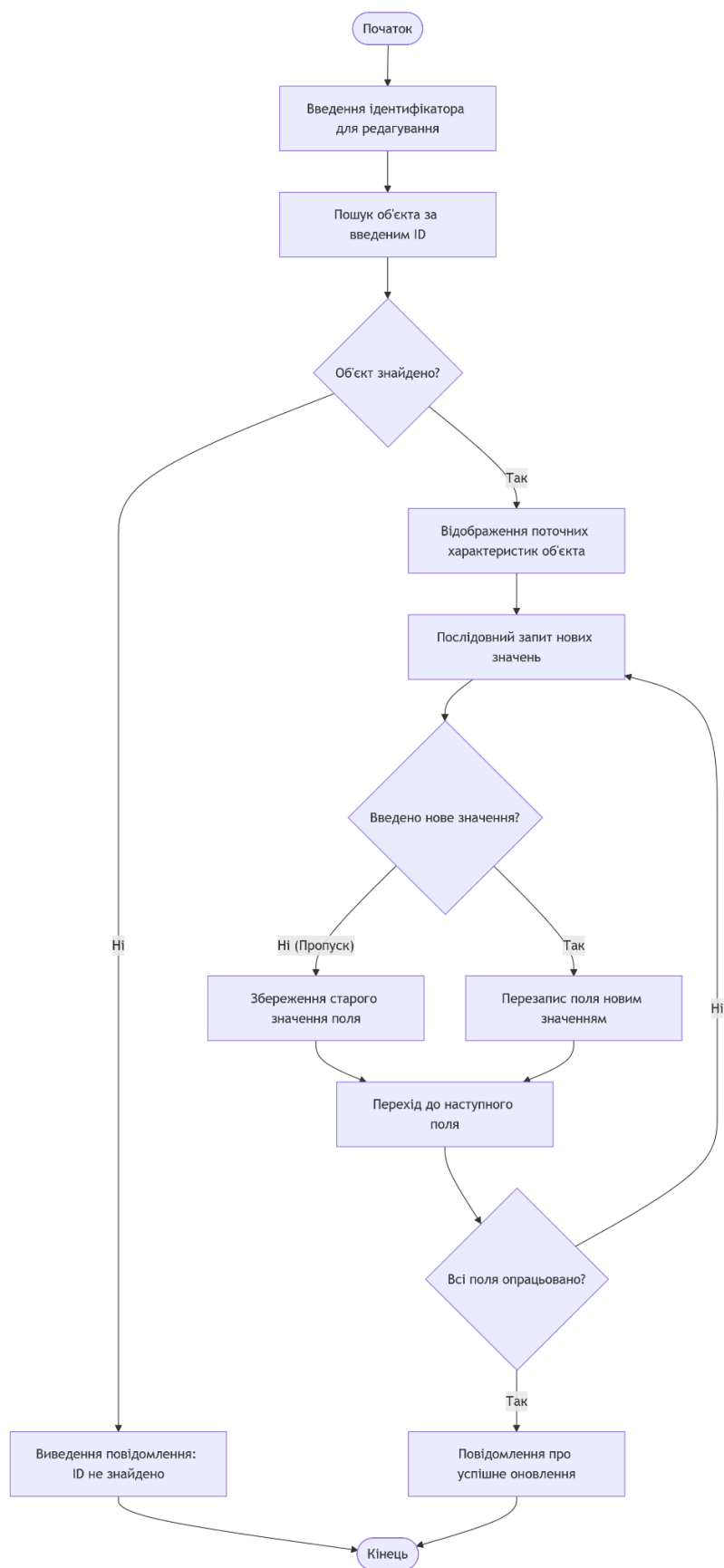


Рисунок 6 – Блок-схема алгоритму редагування існуючої позиції

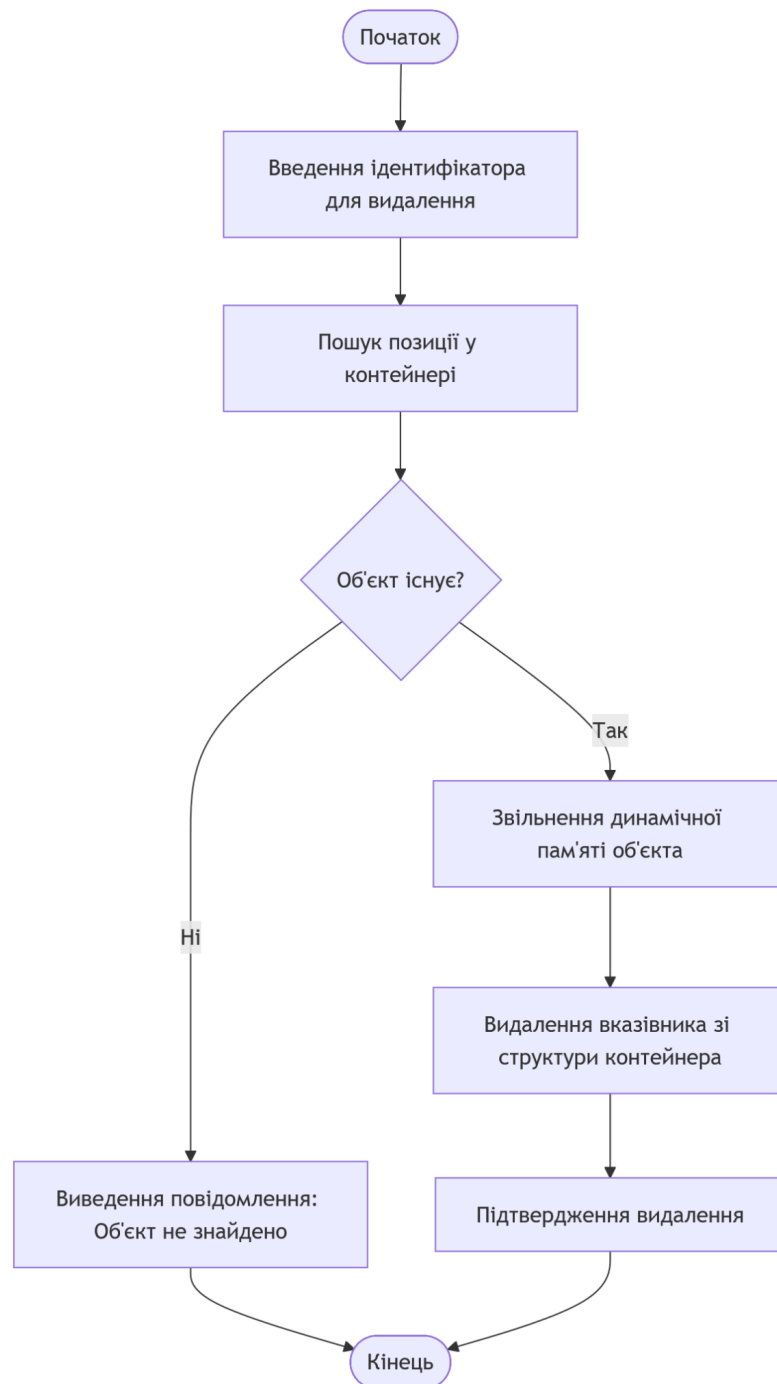


Рисунок 7 – Блок-схема алгоритму безпечного видалення об'єкта

Узагальнені блок-схеми алгоритмів управління базою облікових записів, що включають процеси виведення списку користувачів, створення нового профілю з розподілом прав та видалення існуючого акаунта, наведено на рис. 8 – 10

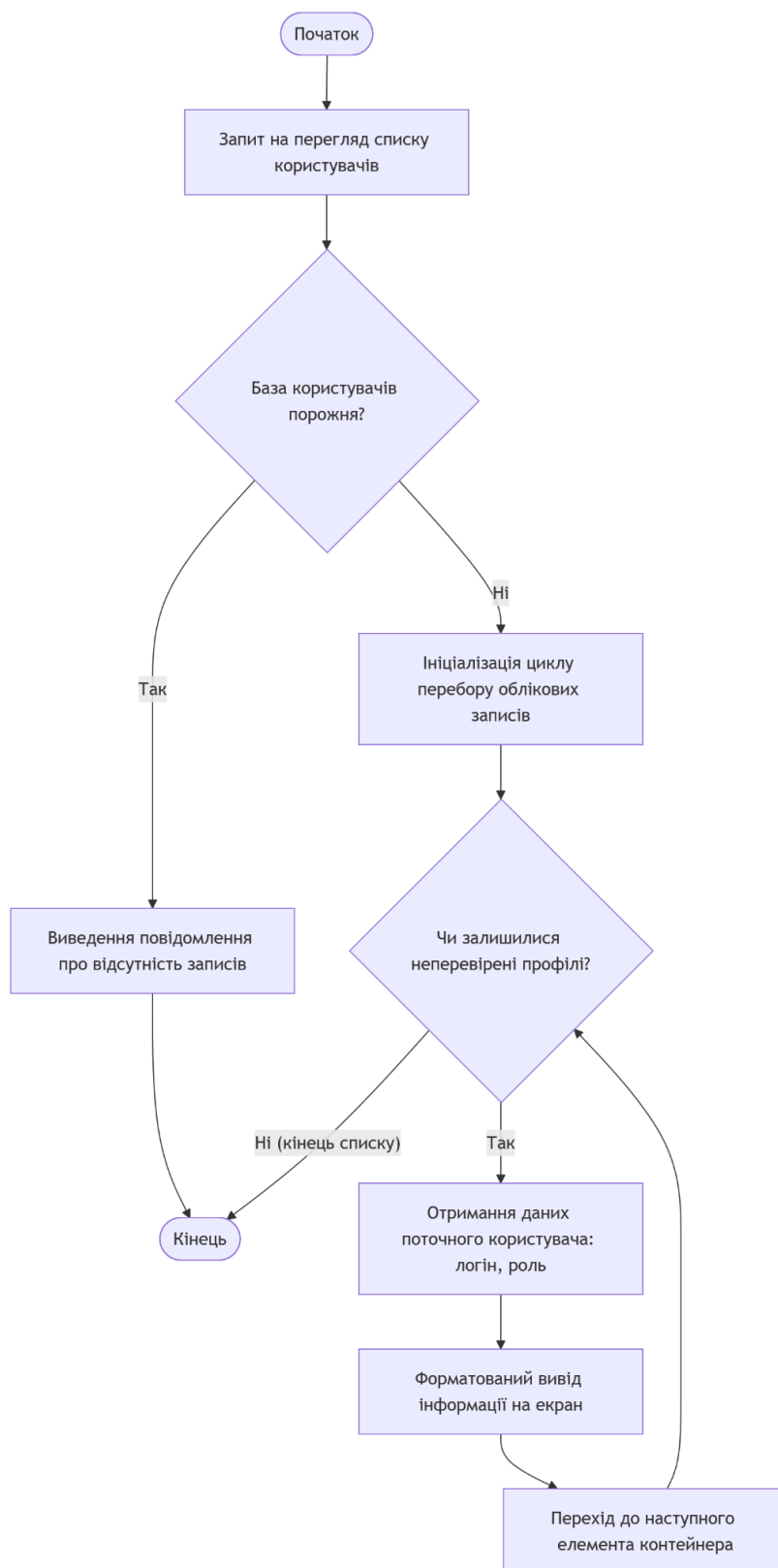


Рисунок 8 – Блок-схема алгоритму перегляду списку облікових записів

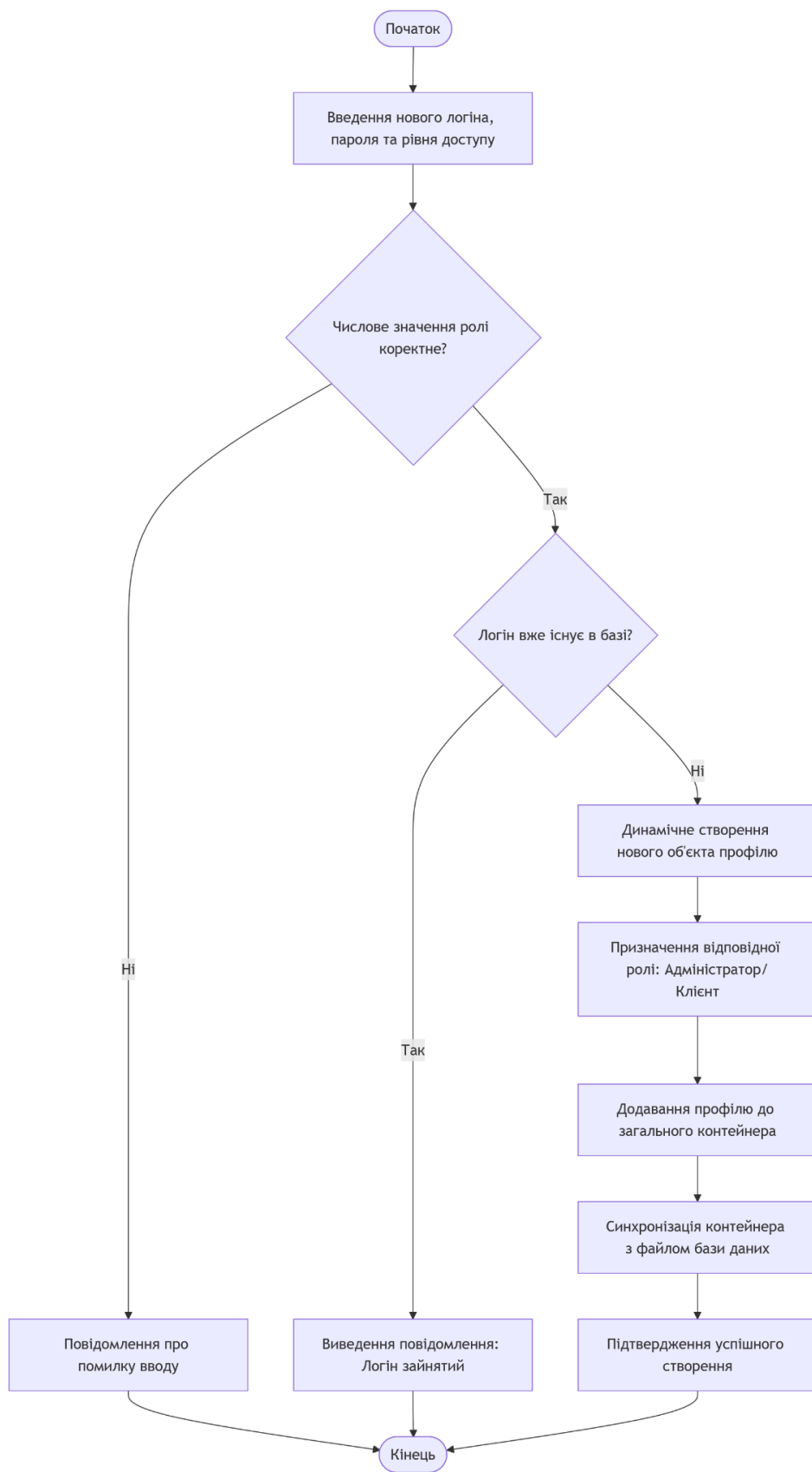


Рисунок 9 – Блок-схема алгоритму створення нового облікового запису адміністратором

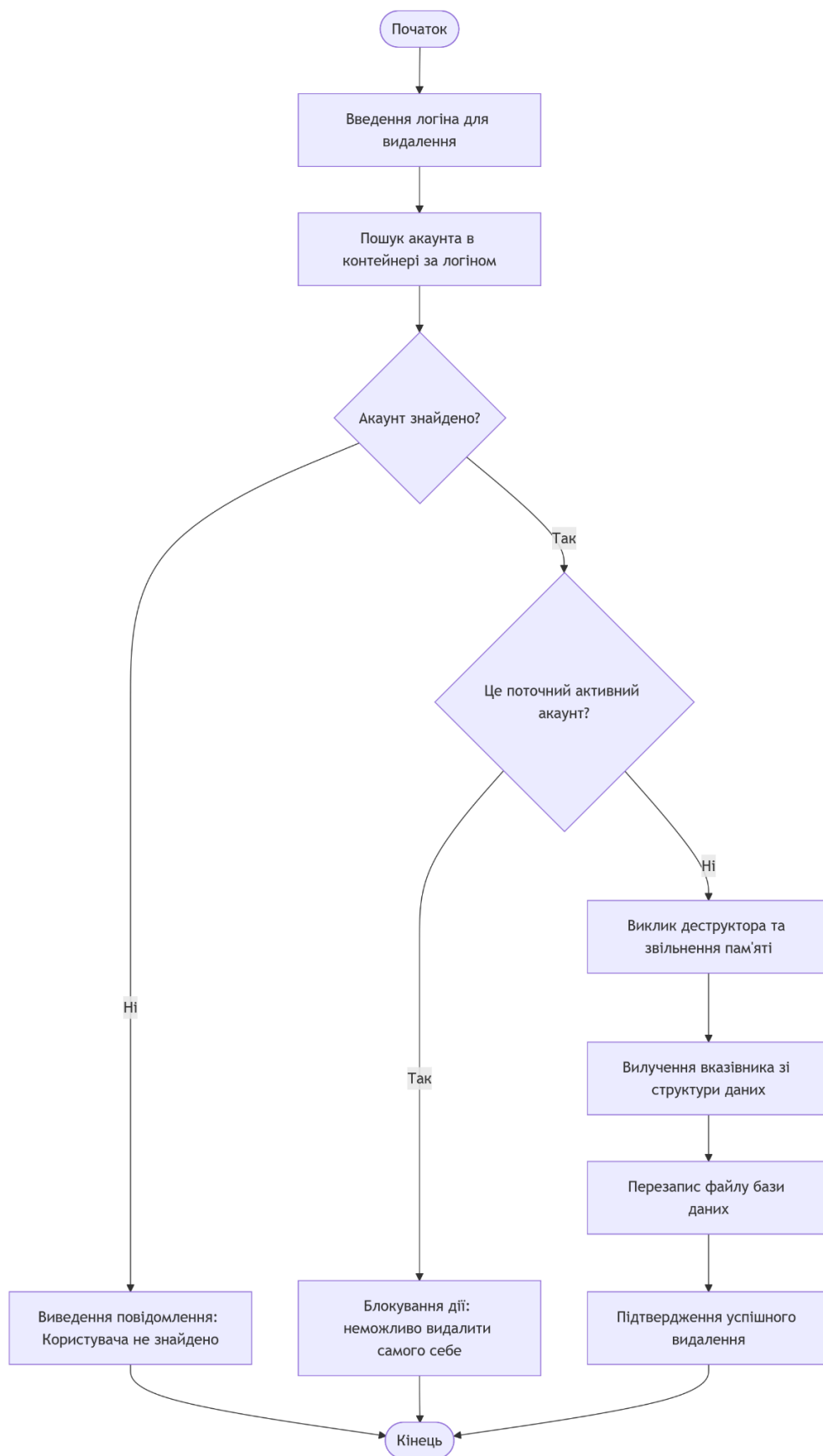


Рисунок 10 – Блок-схема алгоритму видалення облікового запису

2.2. Проєктування та обґрунтування структури класів

Робота розробленого програмного забезпечення реалізується наступними класами:

- 1) MenuItem
- 2) Dish
- 3) Drink
- 4) User
- 5) UserManager
- 6) RestaurantManager
- 7) Order
- 8) IEntity

В табл 8-15 подано документацію класів

Таблиця 8 – Специфікація та структура класу *MenuItem*

Характеристика		Опис
Назва класу		MenuItem
Опис класу		Абстрактний базовий клас, що виступає фундаментальною сутністю для представлення будь-якої позиції в електронному меню закладу та визначає єдиний інтерфейс для всіх нащадків
Залежності від інших класів		Реалізує (успадковує) чисто абстрактний інтерфейс IEntity
Опис атрибутів (полів)		int id - унікальний числовий ідентифікатор позиції в базі даних. std::string name - текстова назва страви або напою. double price - вартість одиниці товару в гривнях. double weight - загальна маса порції або об'єм подачі продукту.
Опис методів		
1	virtual void Display() = 0	Чисто віртуальний метод для форматowanego виведення інформації про об'єкт на екран.
2	virtual std::string ToCsv() const	Віртуальна функція серіалізації полів об'єкта в текстовий рядок для збереження у файлі.
3	GetId()	Метод доступу для зчитування ідентифікатора.
4	GetName()	Метод доступу для отримання назви позиції.
5	GetPrice()	Метод доступу для отримання вартості.

6	virtual ~MenuItem()	Віртуальний деструктор, що гарантує коректне та безпечне очищення пам'яті при роботі з поліморфними об'єктами.
---	---------------------	--

Таблиця 9 – Специфікація та структура класу *Dish*

Характеристика		Опис
Назва класу		Dish
Опис класу		Похідний клас (клас-спадкоємець), який розширює базову модель MenuItem характеристиками, що притаманні суто кулінарним стравам кухонного приготування.
Залежності від інших класів		Успадковує властивості та інтерфейс абстрактного класу MenuItem (відношення відкритого наслідування / public inheritance).
Опис атрибутів (полів)		std::string category - текстова категорія страви int preparationTime - очікуваний час приготування страви шеф-кухарем (вимірюється в хвилинах).
Опис методів		
1	Display()	Перевизначений поліморфний метод виведення повної інформації про страву з урахуванням її категорії та часу готовності.
2	ToCsv()	Перевизначена функція формування спеціалізованого CSV-рядка для збереження даних страви на диску.
3	GetPreparationInfo()	Метод генерації інформаційного рядка для кухарів ресторану.
4	isFastFood()	Логічна функція перевірки страви на мінімальний час очікування.
5	RequiresLongPreparation()	Логічна функція-застереження про тривалий час приготування для складних замовлень.

Таблиця 10 – Специфікація та структура класу *Drink*

Характеристика		Опис
Назва класу		Drink
Опис класу		Похідний клас (спадкоємець), який розширює базову абстрактну модель MenuItem характеристиками, специфічними для барної продукції та напоїв.
Залежності від інших класів		Успадковує властивості та інтерфейс абстрактного класу MenuItem (відношення відкритого наслідування / public inheritance).
Опис атрибутів (полів)		double volume - чистий об'єм рідини в тарі (в літрах).

		bool isAlcoholic - логічний прапорець, що вказує на наявність алкоголю в напої.
Опис методів		
1	Display()	Перевизначений поліморфний метод виведення повної інформації про напій.
2	ToCsv()	Функція формування CSV-рядка для запису об'єкта у файл.
3	NeedsIdCheck()	Логічний метод для ініціалізації перевірки вікових обмежень покупця.
4	CalculateLiterPrice()	Розрахунок відносної вартості напою в перерахунку на один літр.
5	IsFamilySize()	Перевірка об'єму на відповідність великій порції

Таблиця 11 – Специфікація та структура класу *User*

Характеристика		Опис
Назва класу		User
Опис класу		Клас інформаційної моделі (Entity), що представляє профіль користувача системи. Інкапсулює облікові дані та визначає рівень привілеїв (роль).
Залежності від інших класів		—
Опис атрибутів (полів)		std::string username - унікальний текстовий логін користувача. std::string password - пароль доступу облікового запису. bool isAdmin - логічний прапорець адміністративних привілеїв (true – адміністратор, false – клієнт).
Опис методів		
1	GetUsername()	Метод зчитування імені користувача.
2	GetPassword()	Метод зчитування пароля для перевірки при авторизації.
3	IsAdmin()	Перевірка статусу прав адміністратора.
4	ChangePassword(const std::string&)	Метод для безпечної модифікації поточного пароля.

Таблиця 12 – Специфікація та структура класу *UserManager*

Характеристика	Опис
Назва класу	UserManager
Опис класу	Клас-контролер, що інкапсулює логіку адміністрування облікових записів, перевірки прав доступу, валідації при реєстрації та управління активною сесією.
Залежності від інших класів	Агрегує об'єкти класу User. Використовує композицію через вектор вказівників std::vector<User*>.

Опис атрибутів (полів)		std::vector<User> users - динамічна колекція вказівників на всі зареєстровані профілі. User* currentUser - вказівник на об'єкт профілю поточної активної робочої сесії.
Опис методів		
1	Login()	Автентифікація та запуск сесії.
2	Logout()	Анулювання вказівника активної сесії.
3	GetLogin()	Безпечне отримання імені активного користувача.
4	RegisterUser()	Реєстрація з валідацією на унікальність.
5	CreateUser()/DeleteUser ()	Адміністративні функції додавання/вилучення профілів.
6	ListUsers()	Виведення списку всіх зареєстрованих профілів.
7	LoadUsersFromFile() / SaveUsersToFile()	Серіалізація/десеріалізація бази користувачів.

Таблиця 13 – Специфікація та структура класу *RestaurantManager*

Характеристика		Опис
Назва класу		RestaurantManager
Опис класу		Головний клас-контролер для адміністрування бази даних електронного меню. Виконує CRUD-операції, сортування та поліморфний пошук позицій.
Залежності від інших класів		Агрегує об'єкти абстрактного типу MenuItem. Використовує поліморфний контейнер std::vector<MenuItem*>.
Опис атрибутів (полів)		std::vector<MenuItem*> item - поліморфний контейнер елементів електронного меню (напоїв та страв).
Опис методів		
1	AddItem()	Додавання нового об'єкта до колекції.
2	RemoveById()	Вилучення позиції за ідентифікатором з очищенням пам'яті.
3	EditItem()	Покрокова модифікація полів існуючого об'єкта.
4	SearchItems()	Регістронезалежний пошук за запитом
5	SortByPrice()	Упорядкування вектора за зростанням вартості.
6	FindById()	Пошук адреси об'єкта в пам'яті.
7	LoadFromFile() / SaveToFile()	Робота з CSV-файлом бази даних.

Таблиця 14 – Специфікація та структура класу *Order*

Характеристика		Опис
Назва класу		Order
Опис класу		Клас бізнес-логіки, який моделює взаємодію клієнта з обраними товарами (цифровий кошик), виконує агрегацію та розрахунок підсумкової вартості.

Залежності від інших класів		Агрегує посилання на існуючі об'єкти типу MenuItem без управління їхнім життєвим циклом (асоціація).
Опис атрибутів (полів)		std::vector<MenuItem*> orderItem - контейнер вказівників на додані до поточного замовлення позиції.
Опис методів		
1	AddItem()	Додавання вибраної позиції до замовлення.
2	CalculateTotal()	Розрахунок сумарної вартості всіх об'єктів у кошику.
3	DisplayOrder()	Формування та виведення фіскального чека на екран.
4	ClearOrder()	Повне очищення контейнера після завершення транзакції.
5	GetItemsCount()	Отримання кількості найменувань у замовленні.
6	IsEmpty()	Перевірка кошика на наявність елементів.

Таблиця 14 – Специфікація та структура класу *IEntity*

Характеристика		Опис
Назва класу		IEntity
Опис класу		Чисто абстрактний базовий клас (інтерфейс), що визначає загальний контракт (набір обов'язкових методів) для всіх сутностей бази даних системи. Забезпечує найвищий рівень абстракції.
Залежності від інших класів		—
Опис атрибутів (полів)		—
Опис методів		
1	virtual void Display() = 0	Чисто віртуальний метод, що зобов'язує всі класи-спадкоємці реалізувати власну логіку виведення даних.
2	virtual std::string ToCsv() const = 0	Чисто віртуальний метод, який гарантує, що кожна сутність зможе серіалізувати себе для запису у файл.
3	virtual ~IEntity() = default	Віртуальний деструктор, критично необхідний для безпечного звільнення пам'яті масивів поліморфних вказівників.

2.3. Опис програмної системи

При запуску програми відбувається автоматичне зчитування баз даних з файлів. Після цього користувач потрапляє на головне меню авторизації. (рис. 11)

```
===== ГОЛОВНЕ МЕНЮ =====  
1. Увійти  
2. Створити нового користувача (Реєстрація)  
3. Допомога (Інструкція)  
0. Вихід  
-----  
Ваш вибір:|
```

Рисунок 11 - Головне меню авторизації та реєстрації

У разі введення неправильного логіна або пароля система виводить відповідне попередження та повертає користувача до меню входу, запобігаючи несанкціонованому доступу. (рис. 12)

```
Логін: Wrong User  
Пароль: 1234  
Неправильний логін або пароль!  
  
===== ГОЛОВНЕ МЕНЮ =====  
1. Увійти  
2. Створити нового користувача (Реєстрація)  
3. Допомога (Інструкція)  
0. Вихід  
-----  
Ваш вибір:
```

Рисунок 12 - Спроба входу з неправильними даними

Після успішної авторизації під обліковим записом без адміністративних привілеїв, система перемикає інтерфейс на меню

клієнта. Тут доступний функціонал для перегляду асортименту та формування замовлення. (Рис. 13)

```
Логін: User
Пароль: user123

Вхід успішний!

--- МЕНЮ КЛІЄНТА ---
1. Переглянути меню страв та напоїв
2. Пошук страв та напоїв
3. Сортування меню за ціною
4. Додати страву до замовлення
5. Розрахувати вартість (Чек)
6. Допомога
0. Вихід
Ваш вибір: |
```

Рисунок 13 - Головне меню клієнта

Користувач має змогу вивести повний перелік страв та напоїв, а також відсортувати їх за зростанням ціни. (рис. 14)


```

Ваш вибір: 3
Меню відсортовано за ціною.

--- СПИСОК СТРАВ ---
ID: 7 | СТРАВА: Картопля фрі (Гарніри) Вага: 150г | Ціна: 65 грн | Час: 10 хв
ID: 9 | СТРАВА: Торт Фондан шоколадний (Десерти) Вага: 120г | Ціна: 90 грн | Час: 12 хв
ID: 6 | СТРАВА: Деруни зі сметаною та грибами (Гарніри) Вага: 300г | Ціна: 110 грн | Час: 15 хв
ID: 10 | СТРАВА: Чізкейк Нью-Йорк (Десерти) Вага: 150г | Ціна: 115 грн | Час: 5 хв
ID: 1 | СТРАВА: Борщ український з пампушками (Перші страви) Вага: 400г | Ціна: 120 грн | Час: 25 хв
ID: 2 | СТРАВА: Солянка м'ясна (Перші страви) Вага: 350г | Ціна: 140 грн | Час: 20 хв
ID: 8 | СТРАВА: Салат Цезар з куркою (Салати) Вага: 280г | Ціна: 165 грн | Час: 15 хв
ID: 4 | СТРАВА: Паста Карбонара (Основні страви) Вага: 350г | Ціна: 180 грн | Час: 15 хв
ID: 5 | СТРАВА: Лосось на грилі (Основні страви) Вага: 180г | Ціна: 290 грн | Час: 25 хв
ID: 3 | СТРАВА: Стейк Рібай з яловичини (Основні страви) Вага: 250г | Ціна: 350 грн | Час: 30 хв

--- СПИСОК НАПОЇВ ---
ID: 11 | НАПІЙ: Кава Еспreso [Б/А] | Вага: 0.03 | Об'єм: 0.03л. | Ціна: 40 грн.
ID: 15 | НАПІЙ: Кока-Кола в пляшці [Б/А] | Вага: 0.5 | Об'єм: 0.5л. | Ціна: 45 грн.
ID: 13 | НАПІЙ: Чай зелений листовий [Б/А] | Вага: 0.5 | Об'єм: 0.5л. | Ціна: 50 грн.
ID: 12 | НАПІЙ: Кава Капучино [Б/А] | Вага: 0.3 | Об'єм: 0.3л. | Ціна: 55 грн.
ID: 14 | НАПІЙ: Лимонад класичний [Б/А] | Вага: 0.4 | Об'єм: 0.4л. | Ціна: 65 грн.
ID: 16 | НАПІЙ: Пиво світле нефільтроване [АЛКО] | Вага: 0.5 | Об'єм: 0.5л. | Ціна: 75 грн.
ID: 17 | НАПІЙ: Вино червоне сухе Мерло [АЛКО] | Вага: 0.15 | Об'єм: 0.15л. | Ціна: 120 грн.
ID: 18 | НАПІЙ: Коктейль Мохіто [АЛКО] | Вага: 0.35 | Об'єм: 0.35л. | Ціна: 140 грн.
ID: 19 | НАПІЙ: Коктейль Апероль Шприц [АЛКО] | Вага: 0.3 | Об'єм: 0.3л. | Ціна: 160 грн.
ID: 20 | НАПІЙ: Віскі односолодовий [АЛКО] | Вага: 0.05 | Об'єм: 0.05л. | Ціна: 220 грн.

```

Рисунок 14 - Відображення відсортованого електронного меню

Для зручності реалізовано систему пошуку. Користувач може вводити назву, категорію або специфічну ознаку (наприклад, "алко"), і система виведе всі відповідні збіги, ігноруючи регістр вводу. (рис. 15)

```

Ваш вибір: 2

===== ПОШУК У МЕНЮ =====
Ви можете шукати за:
- Назвою (напр. 'борщ' або 'сік')
- Категорією страв (напр. 'десерти')
- Вмістом алкоголю для напоїв ('алко' або 'б/а')
-----
Введіть запит: алко

--- РЕЗУЛЬТАТИ ПОШУКУ: "алко" ---
ID: 16 | НАПІЙ: Пиво світле нефільтроване [АЛКО] | Вага: 0.5 | Об'єм: 0.5л. | Ціна: 75 грн.
ID: 17 | НАПІЙ: Вино червоне сухе Мерло [АЛКО] | Вага: 0.15 | Об'єм: 0.15л. | Ціна: 120 грн.
ID: 18 | НАПІЙ: Коктейль Мохіто [АЛКО] | Вага: 0.35 | Об'єм: 0.35л. | Ціна: 140 грн.
ID: 19 | НАПІЙ: Коктейль Апероль Шприц [АЛКО] | Вага: 0.3 | Об'єм: 0.3л. | Ціна: 160 грн.
ID: 20 | НАПІЙ: Віскі односолодовий [АЛКО] | Вага: 0.05 | Об'єм: 0.05л. | Ціна: 220 грн.

```

Рисунок 15 - Робота функції універсального пошуку

Процес формування замовлення відбувається шляхом введення ідентифікатора (ID) товару. При додаванні об'єктів система інформує клієнта про специфіку обраної позиції (наприклад, попереджає про тривалий час приготування страви або необхідність перевірки документів для алкогольних напоїв). (Рис. 16)

```

--- МЕНЮ КЛІЄНТА ---
1. Переглянути меню страв та напоїв
2. Пошук страв та напоїв
3. Сортуння меню за ціною
4. Додати страву до замовлення
5. Розрахувати вартість (Чек)
6. Допомога
0. Вихід
Ваш вибір: 4
Введіть ID позиції для замовлення: 16

[УВАГА]: Напій алкогольний! Перевірте документи!
[Інфо] Ціна за 1 літр: 150 грн.
Додано: Пиво світле нефільтроване. Товарів у кошику: 1 шт.

```

Рисунок 16 - Додавання позицій до кошика замовлення

Після завершення вибору клієнт може перейти до розрахунку, де формується підсумковий чек. (Рис. 17)

```

--- МЕНЮ КЛІЄНТА ---
1. Переглянути меню страв та напоїв
2. Пошук страв та напоїв
3. Сортуння меню за ціною
4. Додати страву до замовлення
5. Розрахувати вартість (Чек)
6. Допомога
0. Вихід
Ваш вибір: 5

--- Поточне замовлення ---
ID: 16 | НАПІЙ: Пиво світле нефільтроване [АЛКО] | Вага: 0.5 | Об'єм: 0.5л. | Ціна: 75 грн.
Разом: 75 грн.

1 - Підтвердити та оплатити замовлення
2 - Очистити кошик
0 - Назад
Вибір: 1
[ІНФО] Створено копію замовлення.
Чек оплачено на суму: 75 грн.

```

Рисунок 17 - Формування фіскального чека та підрахунок вартості

Якщо авторизація відбувається під акаунтом з правами адміністратора, відкривається розширене меню (CRUD-панель), що дозволяє редагувати базу даних та управляти користувачами. (Рис. 18-20)

```
=== ПАНЕЛЬ АДМІНІСТРАТОРА ===  
1. Вивести список страв та напоїв  
2. Управління меню  
3. Управління акаунтам користувачів  
0. Вихід та збереження  
Ваш вибір: |
```

Рисунок 18 - Головне меню підсистеми адміністратора

```
--- УПРАВЛІННЯ МЕНЮ ---  
1. Додати нову страву  
2. Додати новий напій  
3. Редагування існуючу позицію  
4. Видалити позицію за ID  
0. Повернутися назад  
Ваш вибір:
```

Рисунок 19 - Панель управління електронним меню (CRUD-операції)

```
--- УПРАВЛІННЯ АКАУНТАМИ ---  
  
1. Список  
2. Створити  
3. Видалити  
0. Назад  
Ваш вибір: |
```

Рисунок 20 - Панель управління обліковими записами користувачів

Адміністратор може додавати нові позиції. Залежно від вибору (Страва чи Напій), програма динамічно запитує різні атрибути для заповнення полів відповідних класів. (Рис. 21-22)

```

-----
      ДОДАВАННЯ НОВОЇ СТРАВИ (ID: 21)
-----
Назва страва: Каррі Тіньового Монарха
Категорія: Гарячі страви
Ціна: 245
Вага: 400
Час приготування: 25
Страву успішно додано!

```

Рисунок 21 - Додавання нової страви до бази даних

```

-----
      ДОДАВАННЯ НОВОГО НАПОЮ (ID: 22)
-----
Назва напою: Енергетик "Дедлайн"
Ціна: 85
Вага (загальна маса подачі): 520
Об'єм (рідина): 0.5
Алкогольний (1/0): 0
Напій успішно додано!

```

Рисунок 22 - Додавання нової страви до бази даних

Редагування існуючих записів здійснюється за їх унікальним ідентифікатором. Якщо адміністратор залишає поле порожнім або вводить значення для пропуску, зберігається попередня інформація. (Рис. 23-24)

```

ID для редагування: 21

--- РЕДАГУВАННЯ ПОЗИЦІЇ ID: 21 ---
(Натисніть Enter, щоб залишити поточне значення)
Назва [Каррі Тіньового Монарха]: Ностальгічний сніданок
Ціна [245] (грн): 150
Вага [400] (г):
Категорія [Гарячі страви]: Сніданок
Час приготування [25] (хв): 10
Дані успішно оновлено.

```

Рисунок 23 - Модифікація існуючих записів

```
ID для видалення: 21
[DESTRUCTOR] Об'єкт 'Ностальгічний сніданок' знищено.
Об'єкт з ID21 видалено з меню.
```

Рисунок 24 - Видалення існуючих записів

Окремим модулем є управління обліковими записами. Адміністратор має право переглядати список усіх користувачів системи, створювати нові профілі із заданням ролі (адміністратор/клієнт) та видаляти існуючі акаунти. (Рис. 25-27)

```
--- Список зареєстрованих користувачів ---
Логін: admin [Admin]
Логін: manager [Admin]
Логін: jinwoo [User]
Логін: satoru [User]
Логін: waiter [User]
Логін: guest [User]
Логін: testuser [User]
```

Рисунок 25 - Виведення списку користувачів

```
Логін: User
Пароль: user123
Роль(1-адмін/0-клієнт): 0
Успішно створено новий акаунт!
```

Рисунок 26 - Створення нового користувача

```
Логін: User
Користувача User успішно видалено.
```

Рисунок 27 - Видалення існуючого користувача

Будь-які зміни в базі даних автоматично зберігаються при коректному виході з програми (через пункт "0. Вихід та збереження").

2.4. Тестування програмної системи

Головною метою етапу тестування є перевірка працездатності розробленого програмного забезпечення, виявлення та усунення потенційних помилок, а також підтвердження того, що система коректно реагує як на правильні, так і на некоректні дії користувача. Для перевірки «Restaurant Management System» застосовувалося функціональне тестування (метод «чорного ящика»), яке включало перевірку всіх модулів системи.

Тест 1. Авторизація користувача: Перевірити механізми захисту доступу та створення нових облікових записів.

Кроки тестування:

- При введенні існуючого логіна (наприклад, admin) та правильного пароля, система успішно ідентифікує користувача, визначає його роль та перенаправляє до відповідного меню.
- При спробі введення неправильного пароля або неіснуючого логіна система блокує доступ і виводить попередження «Неправильний логін або пароль!».
- Під час спроби реєстрації нового користувача з логіном, який вже існує в базі (users.txt), спрацьовує валідація, і система видає повідомлення «Користувач з таким логіном вже існує!».

```
===== ГОЛОВНЕ МЕНЮ =====  
1. Увійти  
2. Створити нового користувача (Реєстрація)  
3. Допомога (Інструкція)  
0. Вихід  
-----  
Ваш вибір:1  
  
Логін: admin  
Пароль: admin123  
  
Вхід успішний!  
  
=== ПАНЕЛЬ АДМІНІСТРАТОРА ===  
1. Вивести список страв та напоїв  
2. Управління меню  
3. Управління акаунтам користувачів  
0. Вихід та збереження  
Ваш вибір: |
```

Рисунок 28

```
Логін: WrongUser  
Пароль: 1234  
Неправильний логін або пароль!  
  
===== ГОЛОВНЕ МЕНЮ =====  
1. Увійти  
2. Створити нового користувача (Реєстрація)  
3. Допомога (Інструкція)  
0. Вихід  
-----  
Ваш вибір:|
```

Рисунок 29

```

===== ГОЛОВНЕ МЕНЮ =====
1. Увійти
2. Створити нового користувача (Реєстрація)
3. Допомога (Інструкція)
0. Вихід
-----
Ваш вибір:2

Введіть новий логін: admin
Введіть новий пароль: admin123
Користувач з таким логіном вже існує!

===== ГОЛОВНЕ МЕНЮ =====
1. Увійти
2. Створити нового користувача (Реєстрація)
3. Допомога (Інструкція)
0. Вихід
-----
Ваш вибір:|

```

Рисунок 30

Тест 2. Валідації вводу та обробки виключень (try-catch): Перевірити відмовостійкість програми при введенні некоректних типів даних.

Кроки тестування:

- У головному меню, де програма очікує введення цифри (наприклад, вибір пункту 1, 2 або 0), користувач навмисно вводить символний рядок або літери.

- Замість аварійного завершення (зациклення), стан потоку `std::cin` перехоплюється блоком `try-catch`. Система генерує виключення `std::invalid_argument`, викликає допоміжну функцію `ClearInput()` для очищення буфера та виводить повідомлення «УВАГА: Введіть число!». Після цього програма продовжує стабільну роботу.


```

===== ГОЛОВНЕ МЕНЮ =====
1. Увійти
2. Створити нового користувача (Реєстрація)
3. Допомога (Інструкція)
0. Вихід
-----
Ваш вибір:asd

УВАГА: Введіть число!

===== ГОЛОВНЕ МЕНЮ =====
1. Увійти
2. Створити нового користувача (Реєстрація)
3. Допомога (Інструкція)
0. Вихід
-----
Ваш вибір:|

```

Рисунок 31

Тест 3. CRUD-операції адміністратора (Додавання та видалення об'єктів): Перевірити коректність маніпуляцій з поліморфною базою даних.

Кроки тестування:

- При виборі опції «Додати нову страву» система послідовно запитує характеристики. Після успішного введення даних об'єкт динамічно створюється в пам'яті, отримує унікальний ID і додається до загального списку.

- При спробі видалити об'єкт за існуючим ID система коректно звільняє пам'ять та видаляє вказівник з вектора. Якщо ввести неіснуючий ID, програма виводить повідомлення про помилку та скасовує операцію, запобігаючи зверненню до порожньої пам'яті.

```

--- УПРАВЛІННЯ МЕНЮ ---
1. Додати нову страву
2. Додати новий напій
3. Редагування існуючу позицію
4. Видалити позицію за ID
0. Повернутися назад
Ваш вибір: 1

-----
      ДОДАВАННЯ НОВОЇ СТРАВИ (ID: 21)
-----
Назва страва: Каррі Тіньового Монарха
Категорія: Гарячі страви
Ціна: 245
Вага: 400
Час приготування: 25
Страву успішно додано!

```

Рисунок 32

```

--- УПРАВЛІННЯ МЕНЮ ---
1. Додати нову страву
2. Додати новий напій
3. Редагування існуючу позицію
4. Видалити позицію за ID
0. Повернутися назад
Ваш вибір: 4

ID для видалення: 34
Об'єкт з ID 34 не знайдено.

--- УПРАВЛІННЯ МЕНЮ ---
1. Додати нову страву
2. Додати новий напій
3. Редагування існуючу позицію
4. Видалити позицію за ID
0. Повернутися назад
Ваш вибір: |

```

Рисунок 33

Тест 4. Бізнес-логіки та поліморфізму: Перевірити коректність маніпуляцій з поліморфною базою даних.

Кроки тестування:

- Клієнт додає до замовлення кілька позицій за їхніми ID. Серед позицій є як звичайні страви (Dish), так і алкогольні напої (Drink).
- При додаванні алкогольного напою система успішно приводить тип базового вказівника до Drink*, перевіряє прапорець isAlcoholic і виводить повідомлення: «[УВАГА]: Напій алкогольний! Перевірте документи!». При додаванні страви виводиться інформація про час її приготування.
- Під час вибору пункту «Розрахувати вартість», об'єкт класу Order коректно підсумовує ціни всіх доданих елементів та виводить фіскальний чек на екран.
- Підтвердити оплату замовлення (кошик має очиститися), після чого знову спробувати вивести чек.

```
--- МЕНЮ КЛІЄНТА ---  
1. Переглянути меню страв та напоїв  
2. Пошук страв та напоїв  
3. Сортуння меню за ціною  
4. Додати страву до замовлення  
5. Розрахувати вартість (Чек)  
6. Допомога  
0. Вихід  
Ваш вибір: 4  
Введіть ID позиції для замовлення: 21  
[Кухня] Час очікування: 25 хв.  
Додано: Каррі Тіньового Монарха. Товарів у кошику: 1 шт.
```

Рисунок 34

```

--- МЕНЮ КЛІЄНТА ---
1. Переглянути меню страв та напоїв
2. Пошук страв та напоїв
3. Сортуння меню за ціною
4. Додати страву до замовлення
5. Розрахувати вартість (Чек)
6. Допомога
0. Вихід
Ваш вибір: 4
Введіть ID позиції для замовлення: 20

[УВАГА]: Напій алкогольний! Перевірте документи!
[Інфо] Ціна за 1 літр: 4400 грн.
Додано: Віскі односолодовий. Товарів у кошику: 2 шт.

```

Рисунок 35

```

--- МЕНЮ КЛІЄНТА ---
1. Переглянути меню страв та напоїв
2. Пошук страв та напоїв
3. Сортуння меню за ціною
4. Додати страву до замовлення
5. Розрахувати вартість (Чек)
6. Допомога
0. Вихід
Ваш вибір: 5

--- Поточне замовлення ---
ID: 20 | НАПІЙ: Віскі односолодовий [АЛКО] | Вага: 0.05 | Об'єм: 0.05л. | Ціна: 220 грн.
ID: 21 | СТРАВА: Каррі Тіньового Монарха (Гарячі страви) Вага: 400г | Ціна: 245 грн | Час: 25 хв
Разом: 465 грн.

1 - Підтвердити та оплатити замовлення
2 - Очистити кошик
0 - Назад
Вибір: |

```

Рисунок 36

```

1 - Підтвердити та оплатити замовлення
2 - Очистити кошик
0 - Назад
Вибір: 1
[ІНФО] Створено копію замовлення.

Чек оплачено на суму: 465 грн.

[ДЕКСТРУКТОР] Об'єкт замовлення знищено.

```

Рисунок 37

```

--- МЕНЮ КЛІЄНТА ---
1. Переглянути меню страв та напоїв
2. Пошук страв та напоїв
3. Сортування меню за ціною
4. Додати страву до замовлення
5. Розрахувати вартість (Чек)
6. Допомога
0. Вихід
Ваш вибір: 5
Кошик порожній!

```

Рисунок 38

Тест 5. Пошук об'єктів: Перевірити, чи коректно працює регістронезалежний пошук у поліморфній базі даних страв та напоїв.

Кроки тестування:

- Перейти до меню клієнта та обрати пошук. Ввести частину назви або категорії (наприклад, «десерт» або «сік»), використовуючи різні регістри літер.
- Ввести запит, який точно не відповідає жодній позиції в базі даних (наприклад, вигаданий набір букв)

```

===== ПОШУК У МЕНЮ =====
Ви можете шукати за:
- Назвою (напр. 'борщ' або 'сік')
- Категорією страв (напр. 'десерти')
- Вмістом алкоголю для напоїв ('алко' або 'б/а')
-----
Введіть запит: десерти

--- РЕЗУЛЬТАТИ ПОШУКУ: "десерти" ---
ID: 9 | СТРАВА: Торт Фондан шоколадний (Десерти) Вага: 120г | Ціна: 90 грн | Час: 12 хв
ID: 10 | СТРАВА: Чізкейк Нью-Йорк (Десерти) Вага: 150г | Ціна: 115 грн | Час: 5 хв

```

Рисунок 39

```

===== ПОШУК У МЕНЮ =====
Ви можете шукати за:
- Назвою (напр. 'борщ' або 'сік')
- Категорією страв (напр. 'десерти')
- Вмістом алкоголю для напоїв ('алко' або 'б/а')
-----
Введіть запит: сік

--- РЕЗУЛЬТАТИ ПОШУКУ: "сік" ---
За вашим запитом "сік" нічого не знайдено.

```

Рисунок 40

Тест 6. Сорткування меню: Перевірити, чи правильно система впорядковує загальний список об'єктів за зростанням їхньої вартості.

Кроки тестування:

- В меню клієнта обрати пункт «Сорткування меню за ціною», після чого вивести список страв та напоїв на екран.

```

--- СПИСОК СТРАВ ---
ID: 7 | СТРАВА: Картопля фрі (Гарніри) Вага: 150г | Ціна: 65 грн | Час: 10 хв
ID: 9 | СТРАВА: Торт Фондан шоколадний (Десерти) Вага: 120г | Ціна: 90 грн | Час: 12 хв
ID: 6 | СТРАВА: Деруни зі сметаною та грибами (Гарніри) Вага: 300г | Ціна: 110 грн | Час: 15 хв
ID: 10 | СТРАВА: Чізкейк Нью-Йорк (Десерти) Вага: 150г | Ціна: 115 грн | Час: 5 хв
ID: 1 | СТРАВА: Борщ український з пампушками (Перші страви) Вага: 400г | Ціна: 120 грн | Час: 25 хв
ID: 2 | СТРАВА: Солянка м'ясна (Перші страви) Вага: 350г | Ціна: 140 грн | Час: 20 хв
ID: 8 | СТРАВА: Салат Цезар з куркою (Салати) Вага: 280г | Ціна: 165 грн | Час: 15 хв
ID: 4 | СТРАВА: Паста Карбонара (Основні страви) Вага: 350г | Ціна: 180 грн | Час: 15 хв
ID: 21 | СТРАВА: Каррі Тіньового Монарха (Гарячі страви) Вага: 400г | Ціна: 245 грн | Час: 25 хв
ID: 5 | СТРАВА: Лосось на грилі (Основні страви) Вага: 180г | Ціна: 290 грн | Час: 25 хв
ID: 3 | СТРАВА: Стейк Рібай з яловичини (Основні страви) Вага: 250г | Ціна: 350 грн | Час: 30 хв

--- СПИСОК НАПОЇВ ---
ID: 11 | НАПІЙ: Кава Еспресо [Б/А] | Вага: 0.03 | Об'єм: 0.03л. | Ціна: 40 грн.
ID: 15 | НАПІЙ: Кока-Кола в пляшці [Б/А] | Вага: 0.5 | Об'єм: 0.5л. | Ціна: 45 грн.
ID: 13 | НАПІЙ: Чай зелений листовий [Б/А] | Вага: 0.5 | Об'єм: 0.5л. | Ціна: 50 грн.
ID: 12 | НАПІЙ: Кава Капучино [Б/А] | Вага: 0.3 | Об'єм: 0.3л. | Ціна: 55 грн.
ID: 14 | НАПІЙ: Лимонад класичний [Б/А] | Вага: 0.4 | Об'єм: 0.4л. | Ціна: 65 грн.
ID: 16 | НАПІЙ: Пиво світле нефільтроване [АЛКО] | Вага: 0.5 | Об'єм: 0.5л. | Ціна: 75 грн.
ID: 17 | НАПІЙ: Вино червоне сухе Мерло [АЛКО] | Вага: 0.15 | Об'єм: 0.15л. | Ціна: 120 грн.
ID: 18 | НАПІЙ: Коктейль Мохіто [АЛКО] | Вага: 0.35 | Об'єм: 0.35л. | Ціна: 140 грн.
ID: 19 | НАПІЙ: Коктейль Апероль Шприц [АЛКО] | Вага: 0.3 | Об'єм: 0.3л. | Ціна: 160 грн.
ID: 20 | НАПІЙ: Віскі односолодовий [АЛКО] | Вага: 0.05 | Об'єм: 0.05л. | Ціна: 220 грн.

```

Рисунок 41

Тест 7. Управління користувачами (Адміністрування): Перевірити, чи може адміністратор переглядати базу профілів та чи захищена система від випадкового самовидалення активної сесії.

Кроки тестування:

- Перейти до підменю управління акаунтами і викликати список усіх зареєстрованих користувачів.
- Спробувати видалити обліковий запис адміністратора, під яким наразі здійснюється робота в програмі (поточну сесію).

```
--- УПРАВЛІННЯ АКАУНТАМИ ---  
1. Список  
2. Створити  
3. Видалити  
0. Назад  
Ваш вибір: 1  
  
--- Список зареєстрованих користувачів ---  
Логін: admin [Admin]  
Логін: manager [Admin]  
Логін: jinwoo [User]  
Логін: satoru [User]  
Логін: waiter [User]  
Логін: guest [User]  
Логін: testuser [User]  
Логін: 0 [User]
```

Рисунок 42

```
--- УПРАВЛІННЯ АКАУНТАМИ ---  
1. Список  
2. Створити  
3. Видалити  
0. Назад  
Ваш вибір: 3  
  
Логін: admin  
  
УВАГА: Ви не можете видаляти власний обліковий запис!
```

Рисунок 43

2.5. Програмні засоби розробки

Розробка програмної системи «Restaurant Management System» виконувалася в інтегрованому середовищі розробки Microsoft Visual Studio. Вибір цього середовища зумовлений наявністю потужного вбудованого компілятора, зручним інструментарієм для зневадження (дебагінгу) та нативною підтримкою розробки консольних застосунків під операційну систему Windows.

Програму реалізовано мовою програмування C++. Використання цієї мови дозволило повною мірою застосувати об'єктно-орієнтовану парадигму: побудувати гнучку ієрархію класів (абстрактний клас MenuItem та його нащадки Dish, Drink), реалізувати поліморфізм для обробки колекцій даних, а також використати механізм ідентифікації типів під час виконання (RTTI) за допомогою оператора `dynamic_cast` при формуванні замовлення.

У процесі розробки використовувалися такі стандартні та системні бібліотеки мови C++:

- `<iostream>` та `<fstream>` — для організації базового консольного введення/виведення та роботи з зовнішніми базами даних (`menu.csv` та `users.txt`);
- `<string>` — для збереження та обробки текстових атрибутів (назв позицій, категорій, логінів та паролів користувачів);
- `<vector>` — як основна структура даних для створення динамічних поліморфних колекцій (контейнерів вказівників `std::vector<MenuItem*>`);
- `<algorithm>` — для реалізації алгоритмів упорядкування, зокрема сортування списку меню за ціною за допомогою функції `std::sort` та лямбда-виразів;

- <stdexcept> — для генерування та перехоплення виключних ситуацій (зокрема `std::invalid_argument`) під час валідації користувацького вводу та захисту програми від аварійного завершення;
- <Windows.h> — платформозалежна бібліотека для налаштування локалізації та коректного відображення кириличних символів у консолі Windows (за допомогою функцій `SetConsoleCP` та `SetConsoleOutputCP`).

Інформація про електронне меню зберігається у форматі CSV. Цей формат був обраний через його легковагованість, зручність програмного парсингу та можливість перегляду чи редагування бази даних за допомогою стандартних табличних процесорів (наприклад, MS Excel). Дані про облікові записи та ролі користувачів системи зберігаються у форматованому текстовому файлі.

ВИСНОВКИ

У ході виконання курсового проєкту «Restaurant Management System» було реалізовано такі завдання:

1. Проаналізовано предметну область, що стосується автоматизації роботи закладу громадського харчування, функціонування електронного меню, обробки замовлень відвідувачів та розмежування прав доступу персоналу. Визначено структуру даних для позицій меню, типи сутностей (страви та напої) та механізм формування цифрового кошика замовлень.
2. Сформовано та описано набір класів, що реалізують об'єктно-орієнтовану архітектуру системи: IEntity (базовий інтерфейс), MenuItem (абстрактний базовий клас), Dish та Drink (моделі страв і напоїв), User (модель облікового запису), а також класи-контролери UserManager (керування користувачами та сесіями), RestaurantManager (адміністрування бази даних меню) та Order (обробка поточного замовлення клієнта).
3. Розроблено основний функціонал програмної системи, який включає: автентифікацію та реєстрацію користувачів із розподілом на ролі (клієнт та адміністратор), поліморфний перегляд позицій меню, їх сортування за ціною, універсальний реєстронезалежний пошук, покрокове додавання, редагування та видалення записів, формування кошика з урахуванням специфіки товарів (перевірка вікових обмежень для алкогольних напоїв та розрахунок часу приготування страв), а також підрахунок підсумкової вартості й виведення фіскального чека.
4. Для зберігання даних обрано формат CSV, який забезпечує легковагованість, простоту читання, редагування та сумісність із зовнішніми інструментами обробки таблиць. Уся інформація про електронне меню зберігається у файлі menu.csv, а облікові записи користувачів та їхні ролі — у users.txt.
5. Розроблено консольний застосунок мовою C++ у середовищі Microsoft Visual Studio з використанням стандартних та системних бібліотек (<iostream>, <fstream>, <vector>, <algorithm>, <string>, <stdexcept>, <Windows.h>). Реалізовано механізми динамічного поліморфізму, серіалізації даних, безпечного вивільнення пам'яті за допомогою віртуальних деструкторів, локалізації консолі та стійкого меню взаємодії з користувачем.

6. Проведено функціональне тестування, яке підтвердило коректність роботи основних модулів програми, правильність обробки виключних ситуацій (перехоплення `std::invalid_argument` та захист від некоректного типу вводу), стійкість системи до критичних помилок (запобігання самовидаленню активного адміністратора або зверненню до `nullptr`) та правильність збереження даних у файли. Продемонстровано поведінку системи в коректних і некоректних сценаріях на прикладі ідентифікації типів за допомогою механізму RTTI та розрахунку чека.

Розроблена система може бути розширена у майбутньому шляхом додавання графічного інтерфейсу користувача (GUI), інтеграції з фіскальними реєстраторами для друку реальних чеків, переходу на реляційну систему керування базами даних (СУБД), впровадження модулів клієнтської лояльності (системи бонусів та знижок) або розробки аналітичної панелі для відстеження статистики продажів та найпопулярніших страв закладу.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

- 1) Stroustrup B. The C++ Programming Language. 4th ed. Boston : Addison-Wesley, 2013. 1368 p.
- 2) Doxygen Manual. Doxygen. URL: <https://www.doxygen.nl/manual/index.html>
- 3) Inheritance in C++. *GeeksforGeeks*. URL: <https://www.geeksforgeeks.org/inheritance-in-c/>
- 4) Learn Object-Oriented Programming (OOP) in C++. *Codecademy*. URL: <https://www.codecademy.com/learn/learn-c-plus-plus>
- 5) Regular expressions library. *cppreference.com*. URL: <https://en.cppreference.com/w/cpp/regex>
- 6) Standard Template Library (STL) documentation. *cplusplus.com*. URL: <https://cplusplus.com/reference/>

ДОДАТКИ

ДОДАТОК А. Скролінг (текст) програми

Програмний модуль «Main»

- Вміст Main.cpp

```

/**
 * @file main.cpp
 * @brief Головний файл програмної системи "Restaurant Management System".
 * * Цей файл містить точку входу в програму (функцію main), реалізацію
 * * головного циклу подій, обробку консольного інтерфейсу, а також логіку
 * * взаємодії між користувачем (клієнтом/адміністратором) та менеджерами системи.
 */

#include <iostream>
#include <string>
#include <limits>
#include <stdexcept>
#include <Windows.h>

#include "UserManager.h"
#include "RestaurantManager.h"
#include "Order.h"

/**
 * @brief Допоміжна функція для очищення буфера введення.
 * * Використовується для скидання прапорців помилок потоку std::cin та
 * * очищення залишкових символів у буфері. Запобігає нескінченному
 * * заиканню програми при введенні текстових символів замість числових значень.
 */
void static ClearInput() {
    std::cin.clear();
    std::cin.ignore((std::numeric_limits<std::streamsize>::max)(), '\n');
}

/**
 * @brief Відображення головного меню адміністратора.
 * * Виводить у консоль перелік доступних дій для облікового запису з правами адміністратора.
 * * Реалізує принцип розмежування прав доступу в системі.
 */
void static ShowAdminMenu() {
    std::cout
        << "\n=== ПАНЕЛЬ АДМІНІСТРАТОРА ===\n"
        << "1. Вивести список страв та напоїв\n"
        << "2. Управління меню\n"
        << "3. Управління акаунтам користувачів\n"
        << "0. Вихід та збереження\n"
        << "Ваш вибір: ";
}

/**
 * @brief Відображення підменю для CRUD-операцій з об'єктами меню.
 * * Виводить опції для додавання нових позицій (страв або напоїв),
 * * їх редагування та видалення з бази даних (MenuItem).
 */
void static ShowMenuManagement() {
    std::cout
        << "\n--- УПРАВЛІННЯ МЕНЮ ---\n"
        << "1. Додати нову страву\n"
        << "2. Додати новий напій\n"
        << "3. Редагування існуючої позицію\n"
        << "4. Видалити позицію за ID\n"
        << "0. Повернутися назад\n"
        << "Ваш вибір: ";
}

/**
 * @brief Виведення довідкової інформації.
 * * Короткий посібник (інструкція) для користувача системи щодо

```

```

* правил навігації, збереження даних та призначення ролей.
*/
void static ShowHelp() {
    std::cout << "\n===== ДОПОМОГА (ІНСТРУКЦІЯ) =====\n"
        << "\n1. Для входу введіть логін та пароль.\n"
        << "\n2. Пункти меню обираються введенням відповідної цифри.\n"
        << "\n3. Адміністратор має право на редагування даних.\n"
        << "\n4. При виході (клавіша 0) дані зберігаються автоматично.\n"
        << "\n===== \n";
}

/**
 * @brief Відображення меню для клієнта (Користувача).
 * * Виводить перелік дій, доступних для звичайного клієнта без адміністративних привілеїв:
 * * перегляд меню, пошук, сортування та робота з кошиком замовлень.
 */
void static ShowUserMenu() {
    std::cout
        << "\n--- МЕНЮ КЛІЄНТА ---\n"
        << "1. Переглянути меню страв та напоїв\n"
        << "2. Пошук страв та напоїв\n"
        << "3. Сортування меню за ціною\n"
        << "4. Додати страву до замовлення\n"
        << "5. Розрахувати вартість (Чек)\n"
        << "6. Допомога\n"
        << "0. Вихід\n"
        << "Ваш вибір: ";
}

/**
 * @brief Головна функція програми.
 * * Відповідає за ініціалізацію кодування консолі, створення об'єктів менеджерів,
 * * завантаження початкових даних з файлів та управління нескінченим циклом подій
 * * (автентифікація та обробка запитів користувачів).
 * * @return int Код завершення програми (0 - успішне виконання).
 */
int main() {
    // Встановлення кодування Windows-1251 для коректної підтримки кирилиці в консолі
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);

    // Ініціалізація керуючих класів (Менеджерів)
    UserManager userMgr;
    RestaurantManager restMgr;
    Order currentOrder;

    // Спроба завантаження бази даних при запуску
    try {
        userMgr.LoadUsersFromFile("users.txt");
        restMgr.LoadFromFile("menu.csv");
    }
    catch (const std::exception& e) {
        std::cerr << "Помилка ініціалізації: " << e.what() << std::endl;
    }

    while (true) {
        int authChoice = -1;

        // --- БЛОК АВТОРИЗАЦІЇ ---
        while (userMgr.GetCurrentUser() == nullptr) {

            std::cout << "\n===== ГОЛОВНЕ МЕНЮ =====\n";
            std::cout << "1. Увійти\n";
            std::cout << "2. Створити нового користувача (Реєстрація)\n";
            std::cout << "3. Допомога (Інструкція)\n";
            std::cout << "0. Вихід\n";
            std::cout << "-----\n";
            std::cout << "Ваш вибір:";

            std::string input;
            std::getline(std::cin, input);

            // Перевірка на порожній рядок
            if (input.empty()) {
                std::cout << "\nУВАГА: Ви нічого не ввели. Спробуйте ще раз!" << std::endl;
            }
        }
    }
}

```

```

        continue;
    }

    // Перевірка на коректність числа
    if (input != "0" && input != "1" && input != "2" && input != "3") {
        std::cout << "\nПомилка: Невірний вибір! Введіть 0, 1, 2 або 3." << std::endl;
        continue;
    }

    // Спроба перетворити рядок на число
    try {
        authChoice = std::stoi(input);
    }
    catch (const std::invalid_argument&) {
        // Відпрацює, якщо ввели літери або символи
        std::cout << "\nУВАГА: Введіть число!" << std::endl;
        continue;
    }

    if (authChoice == 0)
        return 0; // Повний вихід з програми
    if (authChoice == 1) {
        // Авторизація користувача
        std::string login, pass;

        std::cout << "\nЛогін: "; getline(std::cin, login);
        std::cout << "Пароль: "; getline(std::cin, pass);

        if (!userMgr.Login(login, pass)) {
            std::cout << "Неправильний логін або пароль!\n";
        }
    }
    else if (authChoice == 2) {
        // Реєстрація користувача
        std::string login, pass;

        std::cout << "\nВведіть новий логін: "; getline(std::cin, login);
        std::cout << "Введіть новий пароль: "; getline(std::cin, pass);

        if (userMgr.RegisterUser(login, pass, false)) {
            std::cout << "Реєстрація успішна! Тепер ви можете увійти.\n";
        }
        else {
            std::cout << "Користувач з таким логіном вже існує!\n";
        }
    }
    else if (authChoice == 3) {
        ShowHelp();
    }
}

int choice = -1;
bool isLogout = false;

std::cout << "\nВхід успішний!\n" << std::endl;

// --- ГОЛОВНИЙ ЦИКЛ ОБРОБКИ ПОДІЙ (після входу) ---
while (!isLogout) {
    try {
        User* user = userMgr.GetCurrentUser();

        // Перевірка ролі та виведення відповідного інтерфейсу
        if (user->IsAdmin()) {
            ShowAdminMenu();
        }
        else {
            ShowUserMenu();
        }

        if (!(std::cin >> choice))
            throw std::invalid_argument("Введіть число!");
        std::cin.ignore();

        if (choice == 0) { // Завершення сесії користувача
            userMgr.Logout();
        }
    }
    catch (const std::invalid_argument&) {
        std::cout << "\nВведіть число!\n";
        continue;
    }
}

```

```

isLogout = true;
std::cout << "Вихід з системи успішний.\n";
break;
}

// --- БЛОК ФУНКЦІОНАЛУ АДМІНІСТРАТОРА ---
if (user->IsAdmin()) {
    switch (choice) {
        case 1: { // Розділений вивід категорій з поліморфним перетворенням
            restMgr.DisplayDishes();
            restMgr.DisplayDrinks();
            break;
        }
        case 2: { // Підменю управління базою MenuItem (CRUD операції)
            int subChoice = -1;
            while (subChoice != 0) {
                ShowMenuManagement();
                std::cin >> subChoice;

                // Збереження змін та вихід з підменю
                if (subChoice == 0) {
                    restMgr.SaveToFile("menu.csv");
                    userMgr.SaveUsersToFile("users.txt");
                    break;
                }

                // Створення об'єкта Dish
                if (subChoice == 1) {
                    int id = restMgr.GenerateNextId();
                    std::string name, cat; double p, w; int t;

                    std::cout << "\n-----"
                        << "\n ДОДАВАННЯ НОВОЇ СТРАВИ (ID: " << restMgr.GenerateNextId() << ") "
                        << "\n-----\n";

                    std::cout << "Назва страва: ";
                    std::cin.ignore(); getline(std::cin, name);

                    std::cout << "Категорія: ";
                    getline(std::cin, cat);

                    std::cout << "Ціна: ";
                    if (!(std::cin >> p))
                        throw std::invalid_argument("Некоректна ціна!");

                    std::cout << "Вага: ";
                    if (!(std::cin >> w))
                        throw std::invalid_argument("Некоректна вага!");

                    std::cout << "Час приготування: ";
                    if (!(std::cin >> t))
                        throw std::invalid_argument("Некоректна час!");

                    restMgr.AddItem(new Dish(id, name, p, w, cat, t));
                    std::cout << "Страву успішно додано!\n";
                }

                // Створення об'єкта Drink
                else if (subChoice == 2) {
                    int id = restMgr.GenerateNextId();
                    std::string name; double p, v, w; bool alc;

                    std::cout << "\n-----"
                        << "\n ДОДАВАННЯ НОВОГО НАПОЮ (ID: " << restMgr.GenerateNextId() << ") "
                        << "\n-----\n";

                    std::cout << "Назва напою: ";
                    std::cin.ignore(); getline(std::cin, name);

                    std::cout << "Ціна: ";
                    if (!(std::cin >> p))
                        throw std::invalid_argument("Некоректна ціна!");

                    std::cout << "Вага (загальна маса подачі): ";
                    if (!(std::cin >> w))

```



```

        throw std::invalid_argument("Некоректна вага!");

    std::cout << "Об'єм (рідина): ";
    if (!(std::cin >> v))
        throw std::invalid_argument("Некоректний об'єм!");

    std::cout << "Алкогольний (1/0): ";
    if (!(std::cin >> alc))
        throw std::invalid_argument("Введіть 1 або 0!");

    restMgr.AddItem(new Drink(id, name, p, v, alc));
    std::cout << "Напій успішно додано!\n";
}
// Редагування існуючих записів за ID
else if (subChoice == 3) {
    int id;
    std::cout << "\nID для редагування: ";

    if (!(std::cin >> id))
        throw std::invalid_argument("Некоректно введено ID!");
    restMgr.EditItem(id);
}
// Видалення об'єкта з бази за ID
else if (subChoice == 4) {
    int id;
    std::cout << "\nID для видалення: ";
    if (!(std::cin >> id))
        throw std::invalid_argument("Некоректно введено ID!");
    restMgr.RemoveById(id);
}
}
choice = -1; // Повернення до головного адмін-меню
break;
}
case 3: { // Підменю управління обліковими записами
    int userChoice = -1;
    while (userChoice != 0) {
        std::cout
            << "\n--- УПРАВЛІННЯ АКАУНТАМИ ---"
            << "\n1. Список"
            << "\n2. Створити"
            << "\n3. Видалити"
            << "\n0. Назад"
            << "\nВаш вибір: ";
        if (!(std::cin >> userChoice))
            throw std::invalid_argument("Некоректно введено ID!");

        // Виведення списку всіх зареєстрованих користувачів
        if (userChoice == 1) {
            userMgr.ListUsers();
        }
        // Адміністративне створення нового користувача з визначенням ролі
        else if (userChoice == 2) {
            std::string n, p;
            int r;
            std::cout << "\nЛогін: "; std::cin >> n;
            std::cout << "Пароль: "; std::cin >> p;

            std::cout << "Роль(1-адмін/0-клієнт): ";

            if (!(std::cin >> r))
                throw std::invalid_argument("Некоректна обрання ролі!");

            userMgr.CreateUser(n, p, r == 1);
        }
        // Видалення користувача за логіном (із перевіркою самовидалення)
        else if (userChoice == 3) {
            std::string l;
            std::cout << "\nЛогін: "; std::cin >> l;
            userMgr.DeleteUser(l);
        }
    }
}
choice = -1;
break;

```

```

    }
    case 0: { // Збереження всіх даних у файли та завершення програми
        restMgr.SaveToFile("menu.csv");
        userMgr.SaveUsersToFile("users.txt");
        return 0;
    }
}
// --- БЛОК ФУНКЦІОНАЛУ КОРИСТУВАЧА ---
else {
    switch (choice) {
        case 1: { // Виведення повного електронного меню
            restMgr.DisplayDishes();
            restMgr.DisplayDrinks();
            break;
        }
        case 2: { // Універсальний пошук (регістронезалежний)
            std::string query;
            std::cout << "\n===== ПОШУК У МЕНЮ =====\n"
                << " Ви можете шукати за:\n"
                << " - Назвою (напр. 'борщ' або 'сік')\n"
                << " - Категорією страв (напр. 'десерти')\n"
                << " - Вмістом алкоголю для напоїв ('алко' або 'б/а')\n"
                << "-----\n"
                << "Введіть запит: ";

            std::cin.ignore(); getline(std::cin, query);
            restMgr.SearchItems(query);
            break;
        }
        case 3: { // Поліморфне сортування колекції за ціною
            restMgr.SortByPrice();
            std::cout << "Меню відсортовано за ціною." << std::endl;
            restMgr.DisplayDishes();
            restMgr.DisplayDrinks();
            break;
        }
        case 4: { // Формування замовлення (додавання об'єкта до кошика)
            int id;
            std::cout << "Введіть ID позиції для замовлення: ";
            std::cin >> id;

            MenuItem* item = restMgr.FindById(id);
            if (item) {
                // RTTI: Якщо знайдений об'єкт є напоєм (Drink)
                if (Drink* dr = dynamic_cast<Drink*>(item)) {
                    if (dr->NeedsIdCheck())
                        std::cout << "\n[УВАГА]: Напій алкогольний! Перевірте документи!\n";

                    std::cout << "[Інфо] Ціна за 1 літр: " << dr->CalculateLiterPrice() << " грн.\n";
                    if (dr->IsFamilySize()) {
                        std::cout << "[Порада] Це велика порція (Family Size), запропонуйте додаткові склянки.\n";
                    }
                }

                // RTTI: Якщо знайдений об'єкт є стравою (Dish)
                if (Dish* ds = dynamic_cast<Dish*>(item)) {
                    std::cout << "[Кухня] " << ds->GetPreparationInfo() << std::endl;

                    if (ds->isFastFood())
                        std::cout << "[Інфо] Ця страва готується швидко.\n";

                    if (ds->RequiresLongPreparation()) {
                        std::cout << "[Інфо] Увага: приготування займе тривалий час.\n";
                    }
                }

                currentOrder.AddItem(item);
                std::cout << "Додано: " << item->GetName() << ". Товарів у кошику: " << currentOrder.GetItemsCount() << " шт." <<
std::endl;
            }
        }
        else
            std::cout << "Помилка: ID не знайдено.\n";
    }
}

```

```

        break;
    }
    case 5: { // Розрахунок вартості (Чек) та оплата
        if (currentOrder.IsEmpty()) {
            std::cout << "Кошик порожній!" << std::endl;
        }
        else {
            currentOrder.DisplayOrder();

            std::cout << "\n1 - Підтвердити та оплатити замовлення";
            std::cout << "\n2 - Очистити кошик";
            std::cout << "\n0 - Назад";
            std::cout << "\nВибір: ";

            int subChoice;
            std::cin >> subChoice;
            if (subChoice == 1) {
                Order archivedOrder = currentOrder;
                std::cout << "\nЧек оплачено на суму: " << currentOrder.CalculateTotal() << " грн." << std::endl;
                currentOrder.ClearOrder();
            }
            else if (subChoice == 2) {
                currentOrder.ClearOrder();
                std::cout << "Кошик очищеною\n";
            }
        }
        break;
    }

    case 6: { // Довідник для користувача
        ShowHelp();
        break;
    }
    case 0: { // Безпечне завершення програми
        return 0;
    }
}
}
catch (const std::exception& e) {
    // Централізована обробка виключних ситуацій циклу
    std::cout << "\nУВАГА: " << e.what() << std::endl;
    ClearInput();
}
}

return 0;
}

```

Програмний модуль «IEntity»

- Вміст IEntity.h

```

#ifndef ENTITY_H
#define ENTITY_H

#include <string>

// using namespace std;

/**
 * @interface IEntity
 * @brief Абстрактний інтерфейс для всіх сутностей системи.
 * * Визначає контракт для класів-нащадків, забезпечуючи уніфікований
 * * спосіб відображення та збереження даних.
 */
class IEntity {
public:
    /**
     * @brief Чистий віртуальний метод для виводу даних у консоль.
     * * Реалізується кожним нащадком індивідуально (Поліморфізм).
     */
    virtual void Display() const = 0;

```

```

/**
 * @brief Чистий віртуальний метод для формування CSV-рядка.
 * Використовується для серіалізації об'єктів при записі у файл.
 * @return Рядок у форматі "значення1,значення2,..."
 */
virtual std::string ToCsv() const = 0;

/**
 * @brief Віртуальний деструктор.
 * Для коректного виклику деструкторів нащадків (Dish, Drink)
 * при видаленні через вказівник на базовий клас.
 */
virtual ~Entity() {}
};

```

```
#endif
```

Програмний модуль «MenuItem»

- Вміст MenuItem.h

```

#ifndef MENUITEM_H
#define MENUITEM_H

#include "Entity.h"
#include <iostream>
#include <string>

/**
 * @class MenuItem
 * @brief Абстрактний базовий клас для всіх позицій меню.
 * Успадковує інтерфейс IEntity та додає загальні атрибути для страв і напоїв.
 */
class MenuItem : public Entity {
protected:
    // Protected дозволяє нащадкам (Dish, Drink) мати прямий доступ
    int id;          ///< Унікальний ідентифікатор
    std::string name; ///< Назва позиції
    double price;    ///< Вартість
    double weight;   ///< Вага або об'єм

public:
    // --- Конструктори та деструктори ---

    /** @brief Конструктор за замовчуванням. */
    MenuItem();

    /** @brief Конструктор з параметрами. */
    MenuItem(int id, std::string name, double price, double weight);

    /** @brief Конструктор копіювання. */
    MenuItem(const MenuItem& other);

    /** @brief Конструктор переміщення. */
    MenuItem(MenuItem&& other) noexcept;

    /** @brief Віртуальний деструктор для забезпечення коректного очищення пам'яті. */
    virtual ~MenuItem();

    // --- Геттери та сеттери (Інкапсуляція) ---

    int GetId() const { return id; }
    std::string GetName() const { return name; }
    double GetPrice() const { return price; }
    double GetWeight() const { return weight; }

    /** @brief Встановлює нову ціну. */
    void SetPrice(double priceValue) { price = priceValue; }

    /** @brief Встановлює нову вагу об'єкта з валідацією. */
    void SetWeight(double w) {
        if (w > 0)
            weight = w;
    }

    // --- Бізнес-логіка класу ---

```

```

/** @brief Перевіряє коректність ціни. */
bool HasPrice() const { return price > 0; }

/** @brief Знижує вартість на заданий відсоток. */
void ApplyDiscount(double percent) {
    if (percent > 0 && percent <= 100)
        price -= price * (percent / 100);
}

/** @brief Розраховує питому вартість продукту. */
double GetPricePerHundredGrams() const {
    return (weight > 0) ? (price / weight) * 100 : 0;
}

/** @brief Категоризація порції за вагою. */
bool IsLargePortion() const {
    return weight > 500;
}

/** @brief Валідація та оновлення назви. */
void UpdateName(const std::string& newName) {
    if (!newName.empty())
        name = newName;
}
};

#endif

```

- Вміст MenuItem.cpp

```

#include "MenuItem.h"
#include <iostream>
#include <utility>

/** @brief Конструктор за замовчуванням. */
MenuItem::MenuItem() : id(0), name("Unknown"), price(0.0), weight(0.0) {}

/** @brief Конструктор з параметрами. */
MenuItem::MenuItem(int id, std::string name, double price, double weight)
    : id(id), name(name), price(price), weight(weight) {}

/** @brief Конструктор копіювання. */
MenuItem::MenuItem(const MenuItem& other)
    : id(other.id), name(other.name), price(other.price), weight(other.weight) {
    std::cout << "[INFO] Викликано конструктор копіювання: " << name << std::endl;
}

/** @brief Конструктор переміщення. */
MenuItem::MenuItem(MenuItem&& other) noexcept
    : id(other.id),
      name(std::move(other.name)),
      price(other.price),
      weight(other.weight) {
    other.id = 0;
    std::cout << "[INFO] Викликано конструктор переміщення" << std::endl;
}

/** @brief Віртуальний деструктор для забезпечення правильного очищення пам'яті. */
MenuItem::~MenuItem() {
    //std::cout << "[DESTRUCTOR] Об'єкт '" << name << "' знищено." << std::endl;
}

```

Програмний модуль «Dish»

- Вміст Dish.h

```

#ifndef DISH_H
#define DISH_H

#include "MenuItem.h"
#include <iostream>
#include <string>

```

```

/**
 * @class Dish
 * @brief Клас-нащадок для представлення кулінарних страв у меню.
 * Реалізує концепцію успадкування, розширюючи базовий клас MenuItem
 * специфічними для їжі атрибутами: категорією та часом приготування.
 */
class Dish : public MenuItem {
private:
    std::string category; ///< Категорія страви (напр., "Десерти", "Гарячі страви")
    int cookTime;         ///< Час приготування страви у хвиликах

public:
    /**
     * @brief Конструктор з параметрами для ініціалізації об'єкта страви.
     * @param id Унікальний ідентифікатор.
     * @param name Назва страви.
     * @param price Вартість.
     * @param weight Вага в грамах.
     * @param cat Текстова категорія.
     * @param time Час приготування (хв).
     * Викликає конструктор базового класу MenuItem для ініціалізації спільних полів.
     */
    Dish(int id, std::string name, double price, double weight, std::string cat, int time)
        : MenuItem(id, name, price, weight), category(cat), cookTime(time) {
    }

    /**
     * @brief Спеціалізоване виведення інформації про страву в консоль.
     * Перевищує (override) віртуальний метод базового класу для відображення
     * розширеного набору даних, включаючи категорію та час очікування.
     */
    void Display() const override {
        std::cout << "ID: " << GetId() << " | СТРАВА: " << GetName() << " (" << category
            << ") Вага: " << GetWeight() << "г | Ціна: "
            << GetPrice() << " грн | Час: " << cookTime << " хв" << std::endl;
    }

    /**
     * @brief Сериалізація даних об'єкта у формат CSV для збереження у файл.
     * Формує рядок, де поля розділені комами. Порядок полів критично важливий
     * для коректного читування методом RestaurantManager::LoadFromFile.
     */
    std::string ToCsv() const override {
        return std::to_string(GetId()) + "," + GetName() + "," + std::to_string(GetPrice()) + ","
            + std::to_string(GetWeight()) + "," + category + "," + std::to_string(cookTime);
    }

    // --- Геттери та сеттери ---

    /**
     * @brief Повертає назву категорії страви.
     */
    std::string GetCategory() const { return category; }

    /**
     * @brief Оновлює категорію страви.
     */
    void SetCategory(const std::string& cat) { category = cat; }

    /**
     * @brief Повертає встановлений час приготування страви.
     */
    int GetCookTime() const { return cookTime; }

    /**
     * @brief Оновлює час приготування з логічною валідацією.
     * Запобігає встановленню від'ємного або нульового часу приготування.
     */
    void SetCookTime(int time) {
        if (time > 0)
            cookTime = time;
    }

    // --- Бізнес-логіка та аналітичні методи ---

```

```

/**
 * @brief Перевіряє, чи вважається страва "швидкою" (Fast Food).
 * @return true, якщо приготування займає менше 15 хвилин.
 */
bool isFastFood() const { return cookTime < 15; }

/**
 * @brief Визначає страви, що потребують тривалої підготовки (складні страви).
 * @return true, якщо приготування займає більше 40 хвилин.
 */
bool RequiresLongPreparation() const { return cookTime > 40; }

/**
 * @brief Формує сервісне повідомлення для офіціанта або клієнта.
 * @return Рядок з інформацією про орієнтовний час очікування.
 */
std::string GetPreparationInfo() const {
    return "Час очікування: " + std::to_string(cookTime) + " хв.";
}
};

#endif

```

Програмний модуль «Drink»

- Вміст Drink.h

```

#ifndef DRINK_H
#define DRINK_H

#include "MenuItem.h"
#include <iostream>
#include <string>

/**
 * @class Drink
 * @brief Клас-нащадок для представлення напоїв у цифровій системі ресторану.
 * Наслідується від базового класу MenuItem. Додає специфічні характеристики
 * рідинних товарів, такі як літраж та наявність алкоголю, що критично
 * для логіки продажів та вікових обмежень.
 */
class Drink : public MenuItem {
private:
    double volume;    ///< Об'єм напою в літрах (напр., 0.33, 0.5, 1.0)
    bool isAlcoholic; ///< Ознака наявності алкоголю (true - алкогольний, false - б/а)

public:
    /**
     * @brief Конструктор з повною ініціалізацією атрибутів напою.
     * @param id Унікальний номер у базі.
     * @param name Назва.
     * @param price Вартість за одиницю.
     * @param weight Загальна вага продукту.
     * @param vol Об'єм напою в літрах.
     * @param alc Статус алкоголю.
     * Передає загальні параметри у батьківський конструктор MenuItem через список ініціалізації.
     */
    Drink(int id, std::string name, double price, double weight, double vol, bool alc)
        : MenuItem(id, name, price, weight, volume(vol), isAlcoholic(alc)) {}

    /**
     * @brief Спеціалізована візуалізація даних про напій у консоль.
     * * Перевизначає віртуальний метод (Поліморфізм). Додає візуальні маркери
     * [АЛКО] або [Б/А], що дозволяє персоналу швидко ідентифікувати тип товару.
     */
    void Display() const override {
        std::cout << "ID: " << GetId()
            << " | НАПІЙ: " << GetName()
            << (isAlcoholic ? " [АЛКО]" : " [Б/А]")
            << " | Вага: " << GetWeight()
            << " | Об'єм: " << volume
            << "л. | Ціна: " << GetPrice() << " грн." << std::endl;
    }
};

/**

```

```

* @brief Сериалізація об'єкта у CSV-рядок для файлового сховища.
* * Гарантує збереження специфічних полів напою в кінці рядка.
* Використовується RestaurantManager для синхронізації бази даних з диском.
*/
std::string ToCsv() const override {
    return std::to_string(GetId()) + "," + GetName() + "," + std::to_string(GetPrice()) + ","
        + std::to_string(GetWeight()) + "," + std::to_string(volume) + "," + std::to_string(isAlcoholic);
}

// --- Геттери та сеттери ---

/**
* @brief Отримання поточного об'єму напою.
*/
double GetVolume() const { return volume; }

/**
* @brief Встановлення нового об'єму з перевіркою коректності.
* Забезпечує цілісність даних (об'єм не може бути від'ємним).
*/
void SetVolume(double v) {
    if (v > 0)
        volume = v;
}

/**
* @brief Отримання статусу алкогольного вмісту.
*/
bool GetIsAlcoholic() const { return isAlcoholic; }

/**
* @brief Зміна статусу алкоголю.
* Використовується адміністратором при редагуванні бази через RestaurantManager::EditItem.
*/
void SetAlcoholic(bool alc) { isAlcoholic = alc; }

// --- Бізнес-логіка ---

/**
* @brief Перевірка необхідності контролю повноліття клієнта.
* @return true, якщо продаж потребує перевірки документів.
*/
bool NeedsIdCheck() const { return isAlcoholic; }

/**
* @brief Автоматичне визначення великих порцій (сімейного формату).
* @return true, якщо об'єм дорівнює або перевищує 1 літр.
*/
bool IsFamilySize() const { return volume >= 1.0; }

/**
* @brief Економічний розрахунок: вартість напою в перерахунку на 1 літр.
* Допомогає клієнту оцінити вигоду покупки різних об'ємів одного продукту.
*/
double CalculateLiterPrice() const {
    return (volume > 0) ? (GetPrice() / volume) : 0;
}
};

```

```
#endif
```

Програмний модуль «Order»

- Вміст Order.h

```

#ifndef ORDER_H
#define ORDER_H

#include <vector>
#include <iostream>
#include <utility>
#include "MenuItem.h"

/**
* @class Order
* @brief Клас для формування замовлення клієнта та розрахунку його вартості.
* Реалізує механізм агрегації об'єктів MenuItem.

```



```

*/
class Order {
private:
    /**
     * @brief Контейнер вказівників на обрані позиції меню.
     * Використовує поліморфізм для зберігання і страв, і напоїв одночасно.
     */
    std::vector<MenuItem*> items;

public:
    // --- Конструктори та деструктори ---

    Order() = default;

    /**
     * @brief Конструктор копіювання для створення дублікату замовлення.
     */
    Order(const Order& other) : items(other.items) {
        std::cout << "[ІНФО] Створено копію замовлення." << std::endl;
    }

    /**
     * @brief Конструктор переміщення для оптимізації роботи з тимчасовими об'єктами.
     */
    Order(Order&& other) noexcept : items(std::move(other.items)) {
        std::cout << "[ІНФО] Замовлення переміщено." << std::endl;
    }

    /**
     * @brief Деструктор класу Order.
     * @note Очищує лише контейнер вказівників. Самі об'єкти MenuItem залишаються в пам'яті,
     * оскільки ними керує RestaurantManager (уникнення помилки подвійного видалення).
     */
    ~Order() {
        //std::cout << "\n[ДЕСТРУКТОР] Об'єкт замовлення знищено." << std::endl;
    }

    // --- Методи управління замовленням (бізнес-логіка) ---

    /** @brief Додає вказану позицію до поточного замовлення. */
    void AddItem(MenuItem* item) {
        if (item)
            items.push_back(item);
    }

    /**
     * @brief Розраховує загальну суму замовлення.
     * @return Сума цін усіх доданих об'єктів.
     */
    double CalculateTotal() const {
        double total = 0;
        for (const auto item : items) {
            total += item->GetPrice();
        }
        return total;
    }

    /** @brief Анулює поточне замовлення (очищення списку). */
    void ClearOrder() {
        items.clear();
    }

    /** @brief Повертає кількість замовлених позицій. */
    int GetItemsCount() const {
        return static_cast<int>(items.size());
    }

    /** @brief Перевіряє наявність позицій у кошику. */
    bool IsEmpty() const {
        return items.empty();
    }

    /**
     * @brief Виводить деталізований чек у консоль.
     * Використовує поліморфний виклик Display() для кожного елемента.

```

```

*/
void DisplayOrder() const {
    std::cout << "\n--- Поточне замовлення ---" << std::endl;
    for (const auto& item : items) {
        item->Display();
    }
    std::cout << "Разом: " << CalculateTotal() << " грн." << std::endl;
}
};

```

```
#endif // ORDER_H
```

Програмний модуль «User»

- Вміст User.h

```

#ifndef USER_H
#define USER_H

#include <string>
#include <iostream>

// using namespace std;

/**
 * @class User
 * @brief Клас для представлення облікового запису користувача системи.
 * Зберігає облікові дані та визначає рівень доступу.
 */
class User {
private:

    std::string username; ///< Логін користувача
    std::string password; ///< Пароль для автентифікації
    bool isAdminRole; ///< Ознака прав адміністратора

public:
    /**
     * @brief Конструктор з параметрами.
     * @param u Логін користувача.
     * @param p Пароль.
     * @param admin Ознака прав адміністратора (за замовчуванням false - звичайний користувач).
     */
    User(std::string u, std::string p, bool admin = false)
        : username(u), password(p), isAdminRole(admin) {}

    /**
     * @brief Конструктор копіювання.
     * Забезпечує коректне копіювання даних акаунта.
     */
    User(const User& other)
        : username(other.username),
          password(other.password),
          isAdminRole(other.isAdminRole) {}

    /**
     * @brief Деструктор класу.
     */
    ~User() {}

    // --- ГЕТТЕРИ (Інкапсуляція даних) ---

    /** @brief Повертає логін користувача. */
    std::string GetUsername() const { return username; }

    /** @brief Повертає пароль (використовується при перевірці входу). */
    std::string GetPassword() const { return password; }

    /** @brief Перевіряє, чи має користувач права адміністратора. */
    bool IsAdmin() const { return isAdminRole; }

    /**
     * @brief Формування рядка для збереження у файл users.txt.
     * Використовує роздільник ':' для надійності збереження.
     * @return Рядок формату "login:password:isAdmin"
     */

```

```

    */
    std::string ToFileString() const {
        return username + ":" + password + ":" + (isAdminRole ? "1" : "0");
    }
};

```

```
#endif
```

Програмний модуль «UserManager»

- Вміст UserManager.h

```

#ifndef USERMANAGER_H
#define USERMANAGER_H

#include <vector>
#include <string>
#include <fstream>
#include <sstream>
#include <iostream>
#include "User.h"

/**
 * @class UserManager
 * @brief Клас-контролер для управління обліковими записами користувачів.
 * Відповідає за автентифікацію (Login), авторизацію (Rights) та збереження даних акаунтів.
 */
class UserManager {
private:
    std::vector<User*> users; ///< Поліморфна колекція зареєстрованих користувачів.
    User* currentUser;      ///< Вказівник на активну сесію (nullptr, якщо не авторизовано).

public:
    // --- КОНСТРУКТОРИ ТА ДЕСТРУКТОРИ ---

    /**
     * @brief Конструктор за замовчуванням.
     */
    UserManager();

    /**
     * @brief Конструктор копіювання для створення дубліката бази користувачів.
     */
    UserManager(const UserManager& other);

    /**
     * @brief Конструктор переміщення для оптимізації передачі даних.
     */
    UserManager(UserManager&& other) noexcept;

    /**
     * @brief Деструктор.
     * Гарантує очищення динамічної пам'яті для всіх об'єктів User.
     */
    ~UserManager();

    // --- СИСТЕМА АВТОРИЗАЦІЇ ---

    std::string GetLogin() const;

    /**
     * @brief Реєструє нового користувача.
     */
    bool RegisterUser(const std::string& login, const std::string password, bool isAdmin);

    /**
     * @brief Виконує вхід користувача в систему.
     * @return true - якщо пара логін/пароль вірна, false - в іншому випадку.
     */
    bool Login(const std::string& login, const std::string& pass);

    /** @brief Скидає вказівник поточного користувача (вихід із системи). */
    void Logout() { currentUser = nullptr; }

    /** @brief Повертає вказівник на поточного авторизованого користувача. */
    User* GetCurrentUser() const { return currentUser; }

```

```
// --- ФУНКЦІЇ АДМІНІСТРАТОРА ---

/** @brief Реєстрація нового користувача в системі. */
void CreateUser(const std::string& login, const std::string& pass, bool admin = false);

/** @brief Видаляє акаунт за логіном (з перевіркою на самовидалення). */
void DeleteUser(const std::string& login);

/** @brief Виводить список користувачів (доступно лише адміністратору). */
void ListUsers() const;

// --- РОБОТА З ФАЙЛАМИ ---

/** @brief Завантаження бази користувачів із текстового файлу. */
void LoadUsersFromFile(const std::string& filename);

/** @brief Синхронізація поточної колекції користувачів із файлом. */
void SaveUsersToFile(const std::string& filename) const;
};
```

```
#endif
```

- Вміст UserManager.cpp

```
#include "UserManager.h"
#include <iostream>
#include <fstream>
#include <sstream>
#include <stdexcept>
#include <algorithm>

/**
 * @brief Конструктор за замовчуванням.
 * Ініціалізує порожній менеджер без авторизованого користувача.
 */
UserManager::UserManager() : currentUser(nullptr) {}

/**
 * @brief Конструктор копіювання.
 * Створює глибоку копію (Deep Copy) списку користувачів, виділяючи нову пам'ять
 * для кожного об'єкта, щоб уникнути конфліктів при видаленні.
 */
UserManager::UserManager(const UserManager& other) : currentUser(nullptr) {
    for (auto u : other.users) {
        users.push_back(new User(*u));
    }
}

/**
 * @brief Конструктор переміщення.
 * Ефективно передає ресурси від одного
 * об'єкта до іншого без зайвого копіювання.
 */
UserManager::UserManager(UserManager&& other) noexcept
    : users(std::move(other.users)), currentUser(other.currentUser) {
    other.currentUser = nullptr;
}

/**
 * @brief Деструктор класу.
 * Проходить по вектору вказівників і звільняє пам'ять,
 * виділену під кожного User.
 */
UserManager::~UserManager() {
    for (auto u : users) {
        delete u;
    }
    //std::cout << "[DESTRUCTOR] UserManager знищено, пам'ять очищена." << std::endl;
}

/**
 * @brief Алгоритм авторизації.
 * Перебирає базу користувачів і порівнює логін/пароль.
 * @return true, якщо знайдено збіг, інакше false.
 */
bool UserManager::Login(const std::string& login, const std::string& pass) {
    for (auto u : users) {
```

```

        if (u->GetUsername() == login && u->GetPassword() == pass) {
            currentUser = u; // Встановлюємо вказівник на активну сесію
            return true;
        }
    }
    return false;
}

/**
 * @brief Реєстрація нового користувача (для адміністратора).
 * Включає валідацію вхідних даних перед створенням об'єкта.
 */
void UserManager::CreateUser(const std::string& login, const std::string& pass, bool admin) {
    if (login.empty() || pass.empty()) {
        throw std::invalid_argument("Логін або пароль не можуть бути порожніми");
    }
    users.push_back(new User(login, pass, admin));
    std::cout << "Успішно створено новий акаунт!" << std::endl;
}

std::string UserManager::GetLogin() const {
    if (this->currentUser == nullptr)
        return "Гість";
    return this->currentUser->GetUsername();
}

/**
 * @brief Реєстрація нового користувача (для клієнта).
 */
bool UserManager::RegisterUser(const std::string& login, const std::string password, bool isAdmin) {
    // Перевірка, чи логін вільний
    for (auto u : users) {
        if (u->GetUsername() == login)
            return false;
    }

    users.push_back(new User(login, password, isAdmin));

    SaveUsersToFile("users.txt");
    return true;
}

/**
 * @brief Видалення користувача за логіном.
 * Використовує ідіому Erase-Remove для безпечного видалення з вектора.
 */
void UserManager::DeleteUser(const std::string& login) {
    // Захист від самовидалення (логічна система безпеки)
    if (currentUser && currentUser->GetUsername() == login) {
        throw std::runtime_error("Ви не можете видаляти власний обліковий запис!");
    }

    // Використання лямбда-виразу для пошуку та видалення об'єкта
    auto it = std::remove_if(users.begin(), users.end(), [&](User* u) {
        if (u->GetUsername() == login) {
            delete u; // Звільняємо пам'ять перед видаленням вказівника
            return true;
        }
    });
    return false;
});

if (it != users.end()) {
    users.erase(it, users.end()); // Видаляємо "сміття" з кінця вектора
    std::cout << "Користувача " << login << " успішно видалено.\n";
}
else {
    std::cout << "Користувача з таким логіном не знайдено.\n";
}
}

/**
 * @brief Виведення списку облікових записів.
 * Допомогає адміністратору бачити структуру доступу в системі.
 */
void UserManager::ListUsers() const {

```

```

std::cout << "\n--- Список зареєстрованих користувачів ---" << std::endl;
for (const auto& u : users) {
    std::cout << "Лорін: " << u->GetUsername()
        << (u->IsAdmin() ? " [Admin]" : " [User]") << std::endl;
}
}

/**
 * @brief Завантаження користувачів із текстового файлу.
 * Використовує роздільник ':' для парсингу рядків.
 */
void UserManager::LoadUsersFromFile(const std::string& filename) {
    std::ifstream file(filename);
    try {
        if (!file.is_open()) {
            // Резервний механізм: якщо бази немає, створюємо root-адміна
            CreateUser("admin", "admin123", true);
            return;
        }

        std::string line;
        while (getline(file, line)) {
            std::stringstream ss(line);
            std::string u, p, isAdminStr;
            // Парсинг рядка формату логін:пароль:роль
            if (getline(ss, u, ':') && getline(ss, p, ':') && getline(ss, isAdminStr)) {
                users.push_back(new User(u, p, isAdminStr == "1"));
            }
        }
    }
    catch (const std::exception& e) {
        std::cerr << "Помилка при роботі з файлом користувачів: " << e.what() << std::endl;
    }
    file.close();
}

/**
 * @brief Збереження бази користувачів у файл.
 * Викликає метод ToFileString() для кожного об'єкта.
 */
void UserManager::SaveUsersToFile(const std::string& filename) const {
    std::ofstream file(filename);
    if (!file.is_open()) {
        throw std::runtime_error("Не вдалося відкрити файл для збереження користувачів.");
    }
    for (const auto u : users) {
        file << u->ToFileString() << std::endl;
    }
    file.close();
}

```

Програмний модуль «RestaurantManager»

- Вміст RestaurantManager.h

```

#ifndef RESTAURANTMANAGER_H
#define RESTAURANTMANAGER_H

#include <vector>
#include <algorithm>
#include <fstream>
#include <iostream>
#include <string>
#include <cctype>

#include "MenuItem.h"
#include "Dish.h"
#include "Drink.h"

// using namespace std;

/**
 * @class RestaurantManager
 * @brief Менеджер-клас для управління колекціями об'єктів меню.
 * * Реалізує бізнес-логіку системи: CRUD-операції, пошук, сортування та фільтрацію.
 * * Відповідає за життєвий цикл об'єктів MenuItem (Dish та Drink).
 */

```

```

class RestaurantManager {
private:
    /**
     * @brief Поліморфна колекція вказівників на MenuItem.
     * Дозволяє зберігати об'єкти різних класів-нащадків в одному контейнері.
     */
    std::vector<MenuItem*> menu; ///< Поліморфна колекція вказівників

public:
    /**
     * @brief Деструктор класу.
     * Забезпечує коректне звільнення динамічної пам'яті для всіх об'єктів меню.
     */
    ~RestaurantManager() {
        for (auto item : menu)
            delete item;
        menu.clear();
    }

    // --- CRUD операції ---

    /** @brief Додає новий об'єкт до меню. */
    void AddItem(MenuItem* item) {
        if (item)
            menu.push_back(item);
    }

    /** @brief Видаляє об'єкт за ідентифікатором. */
    void RemoveById(int id);

    /** @brief Пошук об'єкта за його унікальним ID. */
    MenuItem* FindById(int id) const;

    /** @brief Редагування існуючих полів об'єкта. */
    void EditItem(int id);

    /** @brief Генерація наступного унікального ID на основі максимального існуючого. */
    int GenerateNextId() const;

    // --- Пошук, сортування, фільтрація ---

    /** @brief Сортує дані за зростанням ціни. */
    void SortByPrice();

    /** @brief Фільтрує записи, виводячи тільки ті, що дешевші за maxPrice. */
    void FilterByPrice(double maxPrice) const;

    /** @brief Пошук об'єкта за точною відповідністю назви. */
    MenuItem* FindByName(const std::string& name) const;

    /** @brief Виводить тільки об'єкти типу Dish.
     * Використовує dynamic_cast для фільтрації типу.
     */
    void DisplayDishes() const;

    /** @brief Виводить тільки об'єкти типу Drink.
     * Використовує dynamic_cast для фільтрації типу.
     */
    void DisplayDrinks() const;

    /** @brief Пошук без урахування регістру.
     * Шукає входження підрядка в назву страви або напою.
     */
    void SearchItems(const std::string& query) const;

    // --- Робота з файлами ---

    /** @brief Сериалізація даних та збереження у файл CSV. */
    void SaveToFile(const std::string& filename) const;

    /** @brief Десериалізація даних з файлу та відновлення об'єктів у пам'яті. */
    void LoadFromFile(const std::string& filename);
};

#endif

```

- Вміст RestaurantManager.cpp

```
#include "RestaurantManager.h"
#include <sstream>
#include <iostream>
#include <algorithm>
#include <limits> // Додано для std::numeric_limits

/**
 * @brief Допоміжний метод для парсингу рядків.
 * Розділяє CSV-рядок на окремі токени для подальшої обробки.
 */
std::vector<std::string> SplitString(const std::string& s, char delimiter) {
    std::vector<std::string> tokens;
    std::string token;
    std::istringstream tokenStream(s);
    while (getline(tokenStream, token, delimiter)) {
        tokens.push_back(token);
    }
    return tokens;
}

/**
 * @brief Завантаження бази даних з файлу.
 * Реалізує відновлення об'єктів (десеріалізація)
 * з урахуванням їхнього типу (Dish/Drink).
 */
void RestaurantManager::LoadFromFile(const std::string& filename) {
    std::ifstream file(filename);
    if (!file.is_open())
        return;

    std::string line;
    while (getline(file, line)) {
        if (line.empty())
            continue;
        auto parts = SplitString(line, ',');
        if (parts.size() < 6)
            continue;

        int id = stoi(parts[0]);
        std::string name = parts[1];
        double price = stod(parts[2]);
        double weight = stod(parts[3]);
        bool isDrink = (parts[5] == "1" || parts[5] == "0") && parts[4].find_first_not_of("0123456789.") == std::string::npos;

        // Логіка розділення об'єктів при читанні
        if (isDrink) {
            // Якщо параметр схожий на дробове число (об'єм), створюємо напій
            AddItem(new Drink(id, name, price, weight, stod(parts[4]), parts[5] == "1"));
        }
        else {
            // В іншому випадку створюємо кулінарну страву
            AddItem(new Dish(id, name, price, weight, parts[4], stoi(parts[5])));
        }
    }
}

/**
 * @brief Видалення елемента за ID.
 * Використовує лямбда-вираз та алгоритм std::remove_if
 * для ефективного очищення контейнера.
 */
void RestaurantManager::RemoveById(int id) {
    auto it = std::remove_if(menu.begin(), menu.end(), [id](MenuItem* m) {
        if (m->GetId() == id) {
            delete m; // Звільнення пам'яті перед видаленням вказівника
            return true;
        }
        return false;
    });
    menu.erase(it, menu.end());

    std::cout << "Об'єкт з ID " << id << " видалено з меню.\n";
}
```



```

else {
    std::cout << "Об'єкт з ID " << id << " не знайдено.\n";
}
}

/**
 * @brief Редагування існуючих даних.
 * Оновлює атрибут об'єкта в пам'яті, використовуючи інкапсульовані методи класу.
 */
void RestaurantManager::EditItem(int id) {
    MenuItem* item = FindById(id);
    if (!item) {
        std::cout << "Об'єкт з ID " << id << " не знайдено!\n";
        return;
    }

    std::string input;
    std::cout << "\n--- РЕДАГУВАННЯ ПОЗИЦІЇ ID: " << id << " ---\n";
    std::cout << "(Нагисніть Enter, щоб залишити поточне значення)\n";

    // Редагування назви
    std::cout << "Назва [" << item->GetName() << "]: ";
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    getline(std::cin, input);
    if (!input.empty()) {
        item->UpdateName(input);
    }

    // Редагування ціни
    std::cout << "Ціна [" << item->GetPrice() << "] (грн): ";
    std::getline(std::cin, input);
    if (!input.empty()) {
        try {
            item->SetPrice(std::stod(input));
        }
        catch (...) {
            std::cout << "Помилка: введено не число. Значення не змінено.\n";
        }
    }

    // Редагування ваги
    std::cout << "Вага [" << item->GetWeight() << "] (г): ";
    std::getline(std::cin, input);
    if (!input.empty()) {
        try {
            item->SetWeight(std::stod(input));
        }
        catch (...) {
            std::cout << "Помилка вводу ваги. Значення не змінено.\n";
        }
    }

    // Специфічні поля для Dish
    if (Dish* d = dynamic_cast<Dish*>(item)) {
        std::cout << "Категорія [" << d->GetCategory() << "]: ";
        std::getline(std::cin, input);
        if (!input.empty())
            d->SetCategory(input);

        std::cout << "Час приготування [" << d->GetCookTime() << "] (хв): ";
        std::getline(std::cin, input);
        if (!input.empty())
            d->SetCookTime(std::stoi(input));
    }

    // Специфічні поля для Drink
    else if (Drink* dr = dynamic_cast<Drink*>(item)) {
        std::cout << "Об'єм [" << dr->GetVolume() << "] (л): ";
        std::getline(std::cin, input);
        if (!input.empty())
            dr->SetVolume(std::stod(input)); // Виправлено: stod замість stoi

        std::cout << "Алкогольний (1-так, 0-ні) [" << dr->GetIsAlcoholic() << "]: ";
        std::getline(std::cin, input);
        if (!input.empty())
            dr->SetAlcoholic(input == "1");
    }
}

```

```

    }
    std::cout << "Дані успішно оновлено.\n";
}

/**
 * @brief Пошук за ідентифікатором.
 * @return Вказівник на об'єкт MenuItem або nullptr.
 */
MenuItem* RestaurantManager::FindById(int id) const {
    for (auto item : menu) {
        if (item->GetId() == id)
            return item;
    }
    return nullptr;
}

/**
 * @brief Генерація нового унікального ID.
 * Гарантує цілісність бази даних при додаванні нових записів.
 */
int RestaurantManager::GenerateNextId() const {
    int maxId = 0;
    for (const auto& item : menu) {
        if (item->GetId() > maxId)
            maxId = item->GetId();
    }
    return maxId + 1;
}

/**
 * @brief Сортвання за ціною.
 * Використовує стандартний алгоритм std::sort
 * з кастомним компаратором.
 */
void RestaurantManager::SortByPrice() {
    std::sort(menu.begin(), menu.end(), [](MenuItem* a, MenuItem* b) {
        return a->GetPrice() < b->GetPrice();
    });
}

/**
 * @brief Фільтрація за ціновим порогом.
 */
void RestaurantManager::FilterByPrice(double maxPrice) const {
    std::cout << "Результати фільтрації (до " << maxPrice << " грн):" << std::endl;
    for (auto item : menu) {
        if (item->GetPrice() <= maxPrice)
            item->Display();
    }
}

/**
 * @brief Пошук без урахування реєстру.
 * Реалізує трансформацію рядків для зручного пошуку користувачем.
 */
void RestaurantManager::SearchItems(const std::string& query) const {

    if (query.empty()) {
        std::cout << "Запит порожній!\n";
        return;
    }

    bool globalFound = false;

    // Приведення запиту до нижнього реєстру для зручності
    auto toLowerUA = [](std::string data) {
        for (int i = 0; i < data.length(); i++) {
            unsigned char c = (unsigned char)data[i];
            if (c >= 'A' && c <= 'Z')
                data[i] = c + 32;
            else if (c >= 192 && c <= 223)
                data[i] = c + 32;
            else if (c == 178) // Обробка української літери 'Г'
                data[i] = 179; // 'г'
        }
    };

```

```

    return data;
};

std::string lowerQuery = toLowerUA(query);

std::cout << "\n--- РЕЗУЛЬТАТИ ПОШУКУ: \'" << query << "\' ---\n";

for (auto item : menu) {
    bool isMatch = false;

    // 1. Пошук за назвою (працює на всіх об'єктах MenuItem)
    std::string lowerName = toLowerUA(item->GetName());
    if (lowerName.find(lowerQuery) != std::string::npos) {
        isMatch = true;
    }

    // 2. Пошук за категорією (тільки для страв)
    if (!isMatch) {
        if (Dish* d = dynamic_cast<Dish*>(item)) {
            std::string lowerCat = toLowerUA(d->GetCategory());
            if (lowerCat.find(lowerQuery) != std::string::npos)
                isMatch = true;
        }
    }

    // 3. Специфічний пошук для напоїв
    // Якщо користувач ввів "алко" або "б/а"
    if (!isMatch) {
        if (Drink* dr = dynamic_cast<Drink*>(item)) {
            if ((lowerQuery == "алко") && dr->GetIsAlcoholic())
                isMatch = true;
            else if ((lowerQuery == "б/а") && !dr->GetIsAlcoholic())
                isMatch = true;
        }
    }

    // Вивід: тільки якщо знайшли хоча б один збіг
    if (isMatch) {
        item->Display();
        globalFound = true;
    }
}
if (!globalFound)
    std::cout << "За вашим запитом \'" << query << "\' нічого не знайдено.\n";
}

MenuItem* RestaurantManager::FindByName(const std::string& name) const {
    for (auto item : menu) {
        if (item->GetName() == name)
            return item;
    }
    return nullptr;
}

/**
 * @brief Збереження у файл (Серіалізація).
 */
void RestaurantManager::SaveToFile(const std::string& filename) const {
    std::ofstream file(filename);
    if (!file.is_open()) {
        throw std::runtime_error("Критична помилка: файл " + filename + " не знайдено!");
    }
    for (const auto& item : menu) {
        file << item->ToCsv() << "\n";
    }
    file.close();
}

/**
 * @brief Виведення страв за допомогою dynamic_cast.
 * Демонструє використання RTTI для фільтрації об'єктів за типом.
 */
void RestaurantManager::DisplayDishes() const {
    std::cout << "\n--- СПИСОК СТРАВ ---\n";
    bool found = false;

```

```

    for (auto item : menu) {
        if (dynamic_cast<Dish*>(item)) {
            item->Display();
            found = true;
        }
    }
    if (!found)
        std::cout << "Страви ще не додані.\n";
}

/**
 * @brief Виведення напоїв за допомогою dynamic_cast.
 */
void RestaurantManager::DisplayDrinks() const {
    std::cout << "\n--- СПИСОК НАПОЇВ ---\n";
    bool found = false;
    for (auto item : menu) {
        if (dynamic_cast<Drink*>(item)) {
            item->Display();
            found = true;
        }
    }
    if (!found)
        std::cout << "Напої ще не додані.\n";
}

```